

Biomedical Computer Vision

Introduction, Data Dimensionality & Format

March 17th, 2025

Virginia Tasso
virginia.tasso@polimi.it



Syllabus:

17.03

- Data dimensionality, image formats
- Image preprocessing

19.03

- Introduction to convolution
- Padding, stride, pooling

24.03

- Convolution-based operation: filtering (denoising, sharpening, edge detection)

27.03

- Introduction to segmentation: thresholding, masking, k-means...
- Post-processing: connected components, erosion, dilation...

31.03

- Introduction to Deep Learning
- Fully Connected Neural Networks, Convolutional Neural Networks

02.04

- NN training and regularization (dropout, weight decay...)

14.04

- Image segmentation with Deep Learning
- UNet

16.04

- CNN and others for other tasks (generation, denoising...)
- Review of the state of the art

Final Project:

Project assignment 31.03.2026

Deadline 15.06.2026

1. Short report reporting what you have done and how
2. Code - invite to a GitHub Repo or a link to a public GitHub Repo

Useful Information and Links

Course material:

- notebooks and images are available at the following link: [BMCV-repo](#)
- slides and recordings are available here: [BMCVCourse](#)

Useful links:

Codecademy (<https://www.codecademy.com/>)

Coursera (Python for everybody Course 1

and 2 <https://www.coursera.org/specializations/python#courses>)

Python tutorials (<https://docs.python.org/3/tutorial/index.html>,
<https://www.learnpython.org/en/Welcome>)

Syllabus:

17.03

- Data dimensionality, image formats
- Image preprocessing

19.03

- Introduction to convolution
- Padding, stride, pooling

24.03

- Convolution-based operation: filtering (denoising, sharpening, edge detection)

27.03

- Introduction to segmentation: thresholding, masking, k-means...
- Post-processing: connected components, erosion, dilation...

31.03

- Introduction to Deep Learning
- Fully Connected Neural Networks, Convolutional Neural Networks

02.04

- NN training and regularization (dropout, weight decay...)

14.04

- Image segmentation with Deep Learning
- UNet

16.04

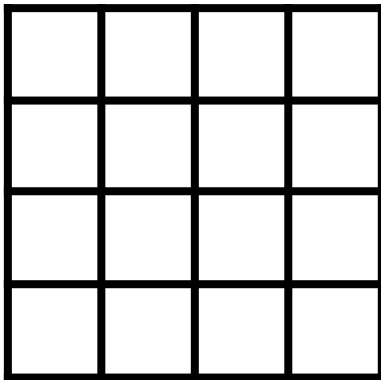
- CNN and others for other tasks (generation, denoising...)
- Review of the state of the art

Data Dimensionality:

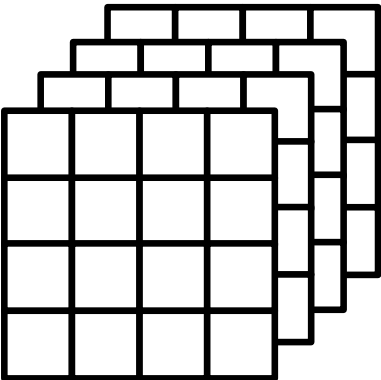
1D



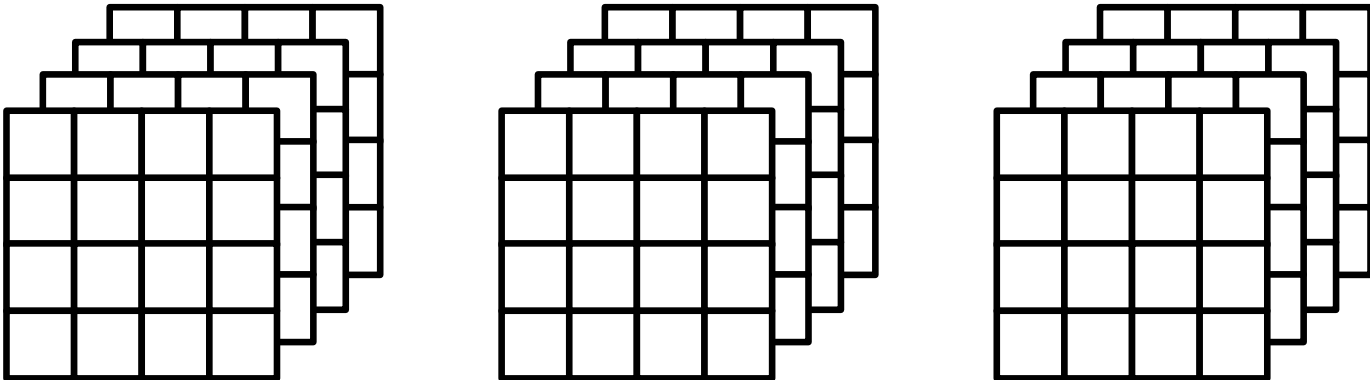
2D



3D

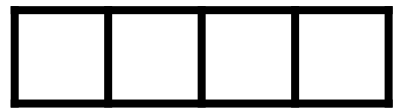


4D

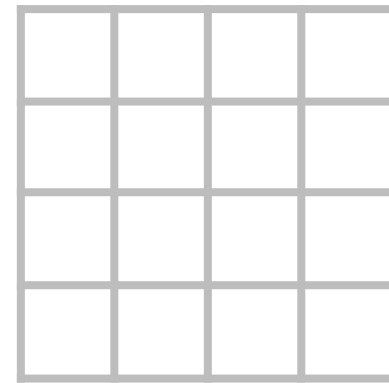


Data Dimensionality:

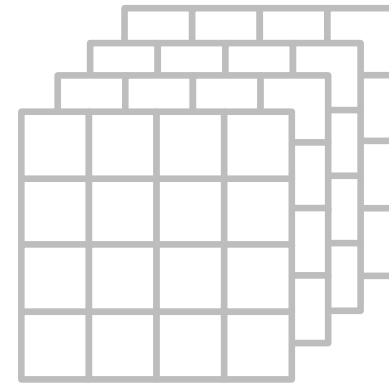
1D



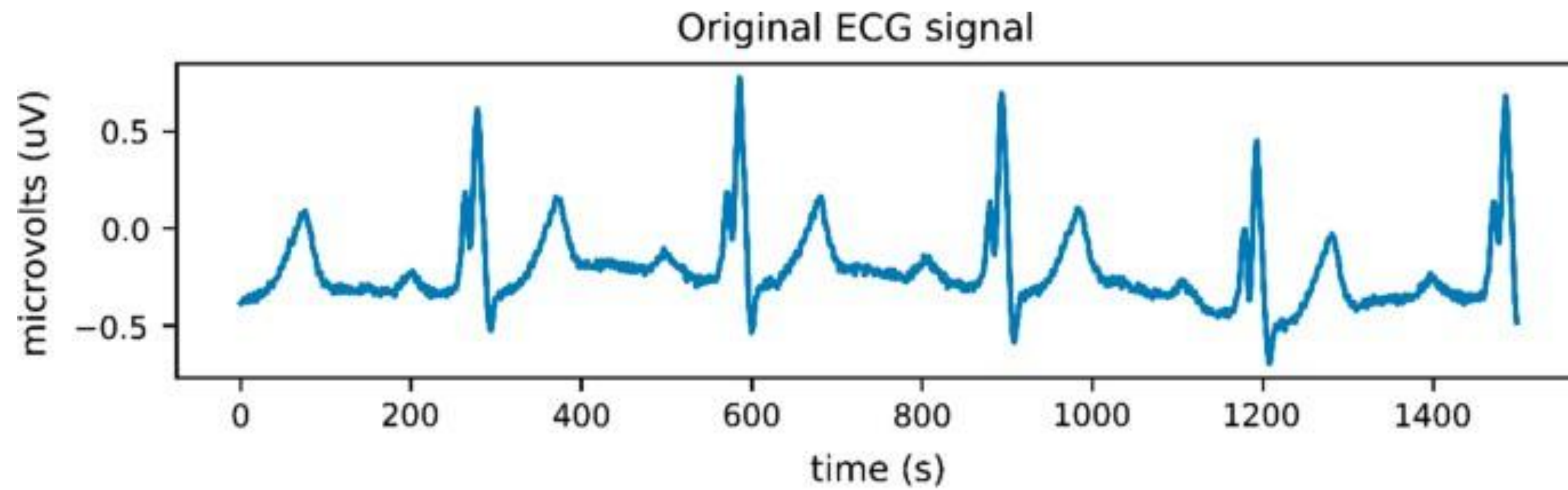
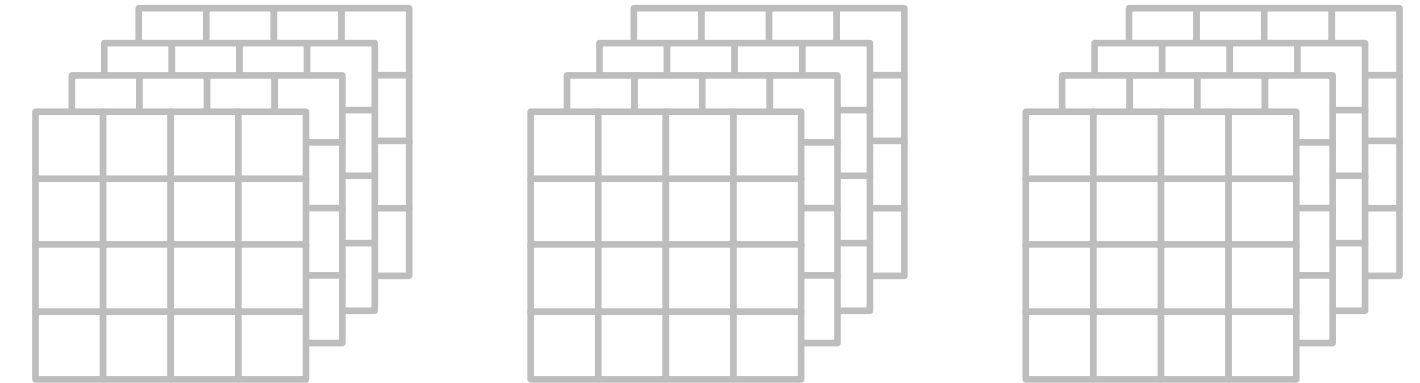
2D



3D



4D

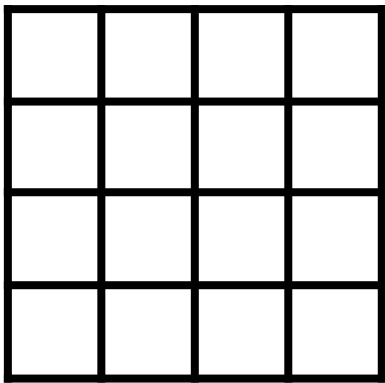


Data Dimensionality:

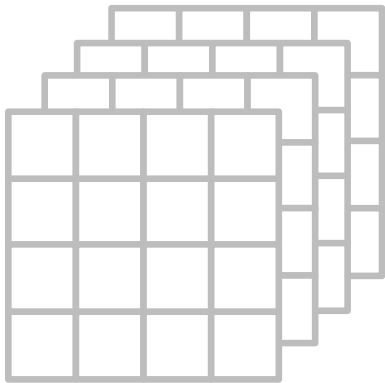
1D



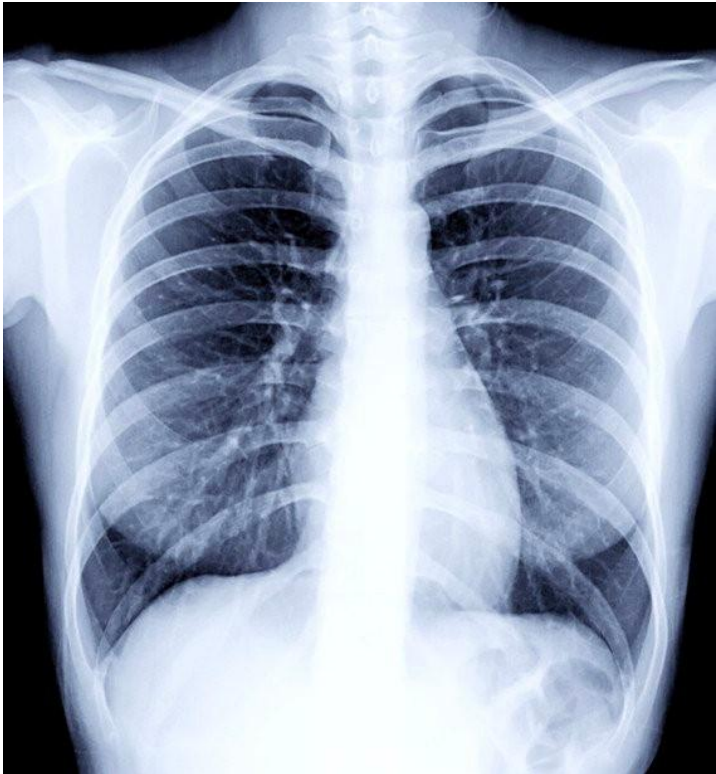
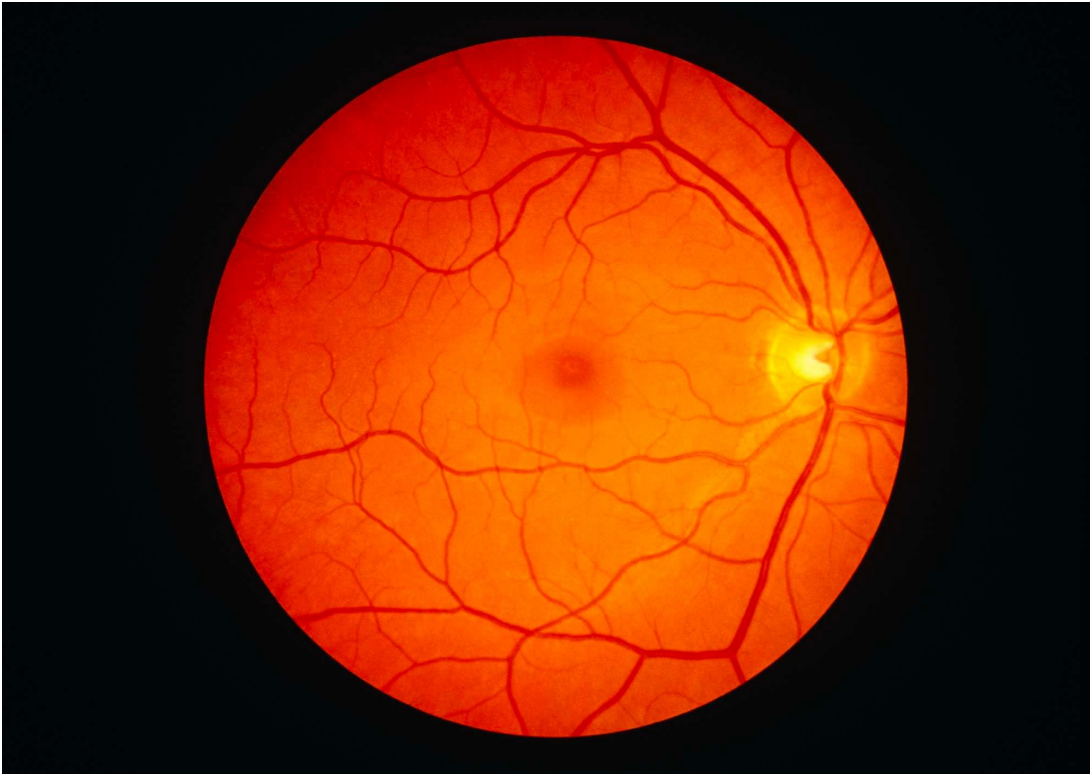
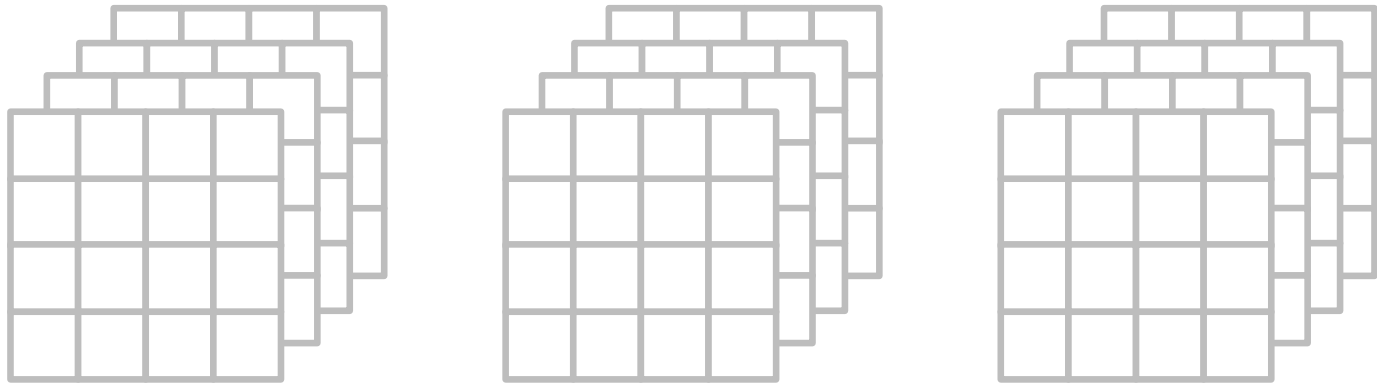
2D



3D



4D

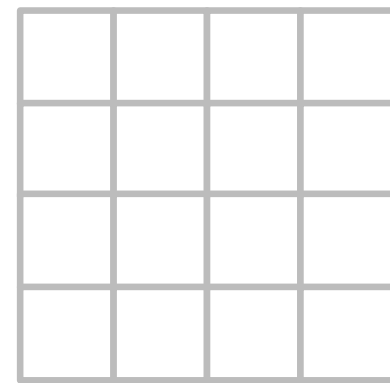


Data Dimensionality:

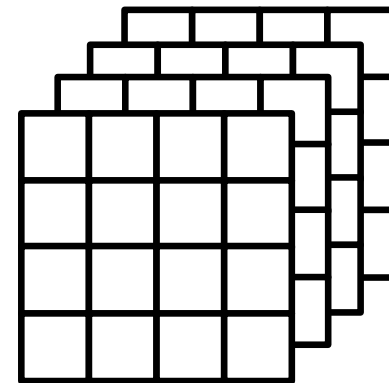
1D



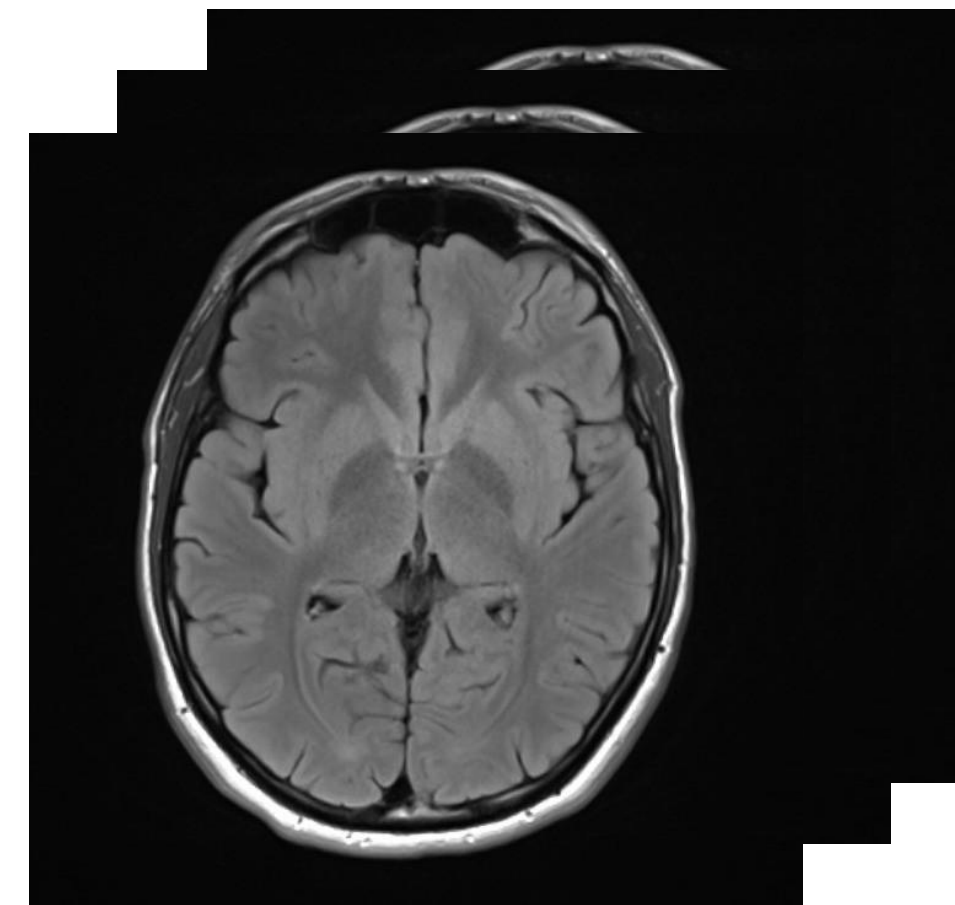
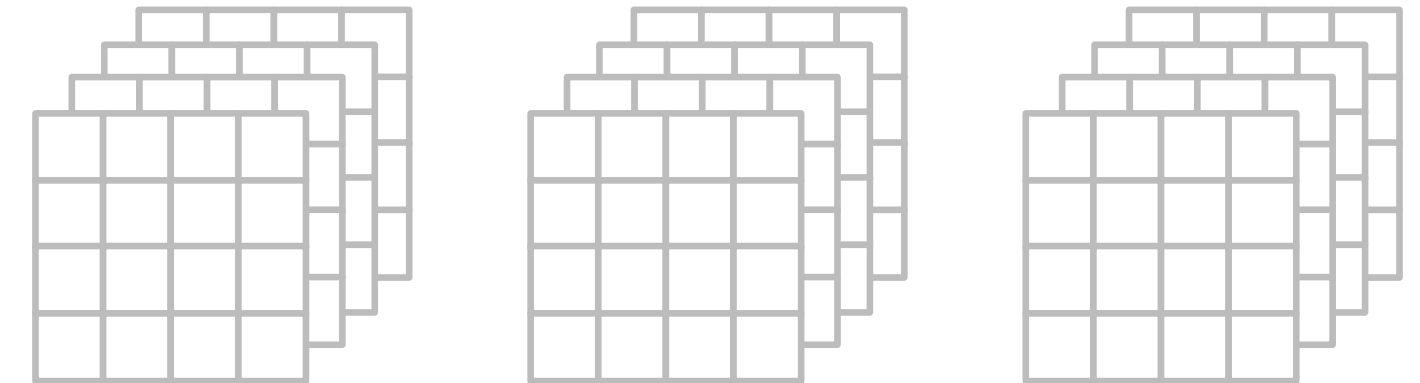
2D



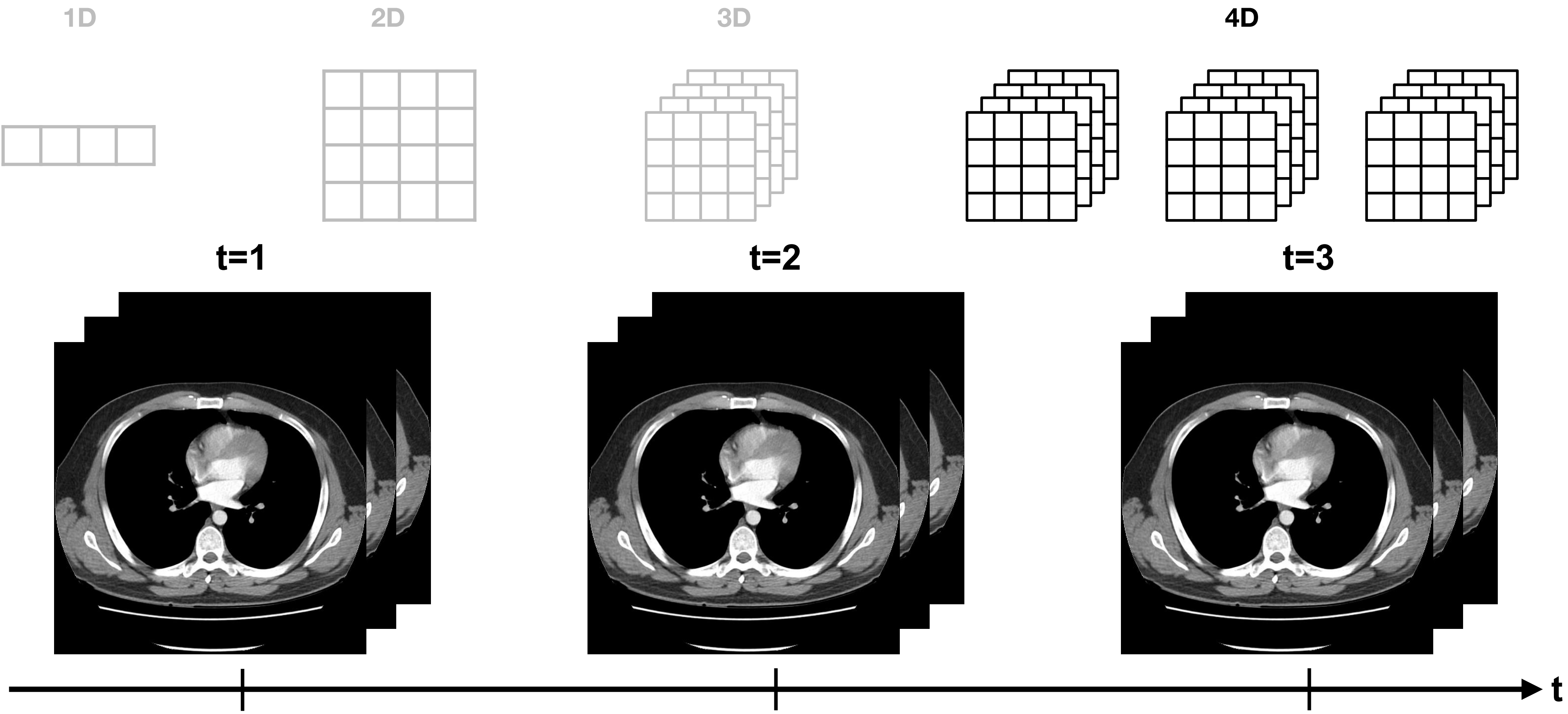
3D



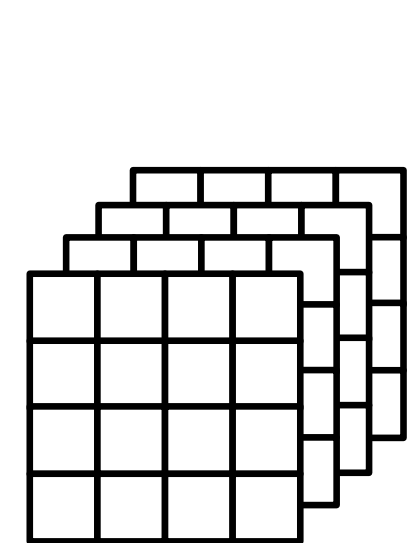
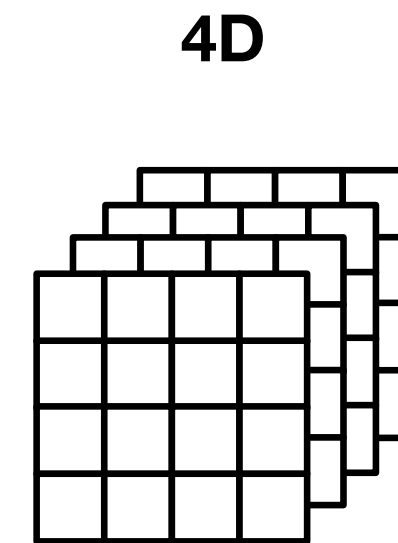
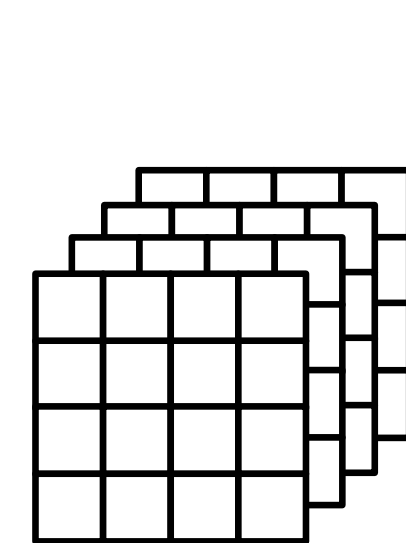
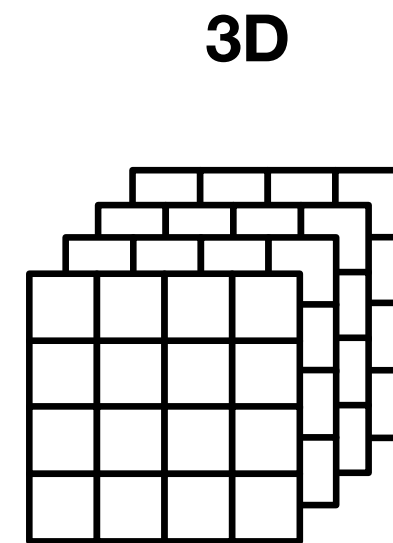
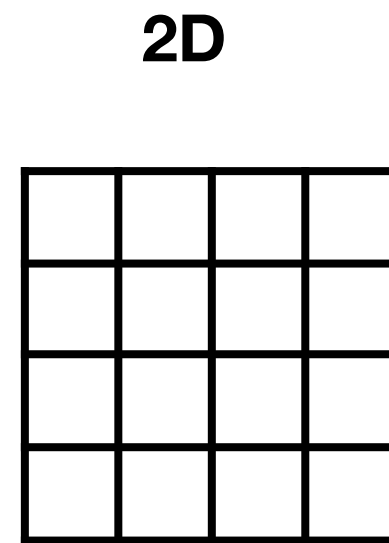
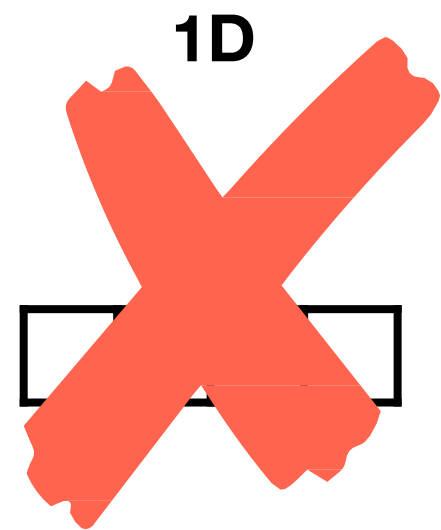
4D



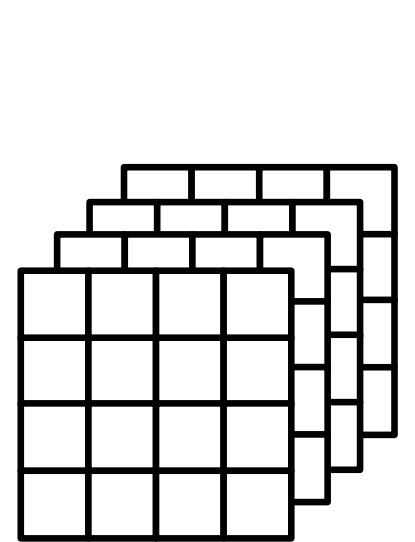
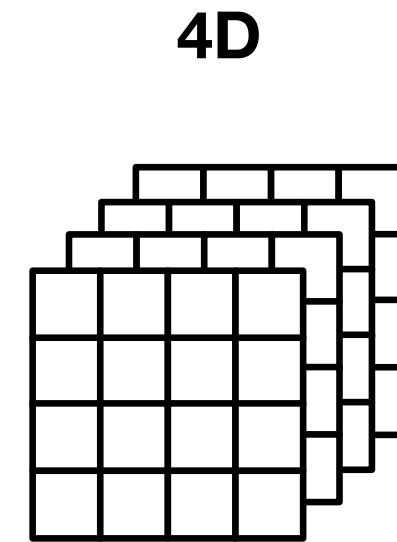
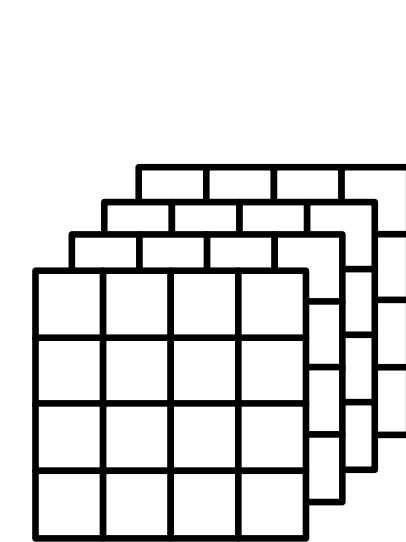
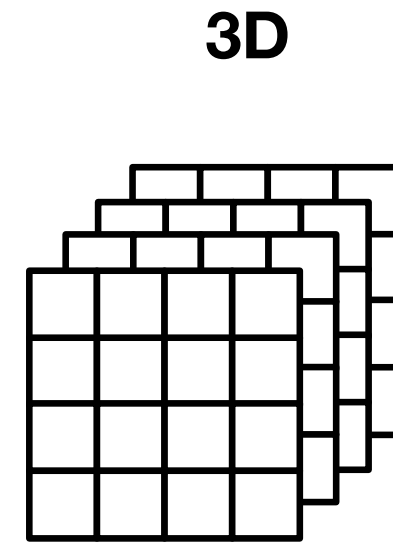
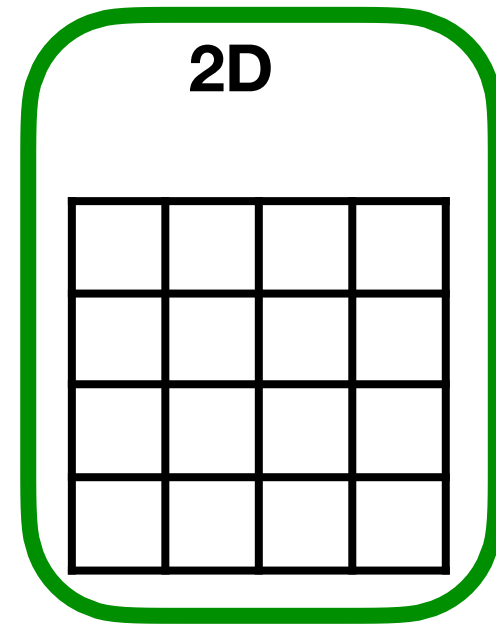
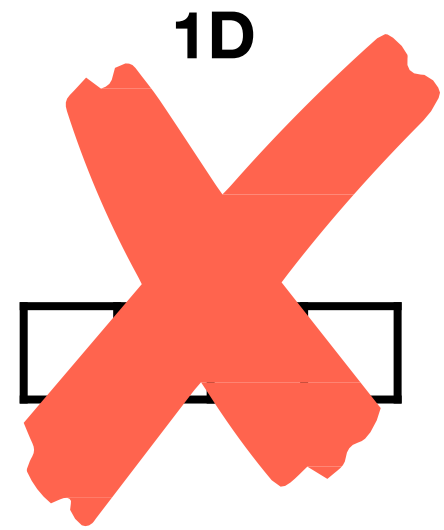
Data Dimensionality:



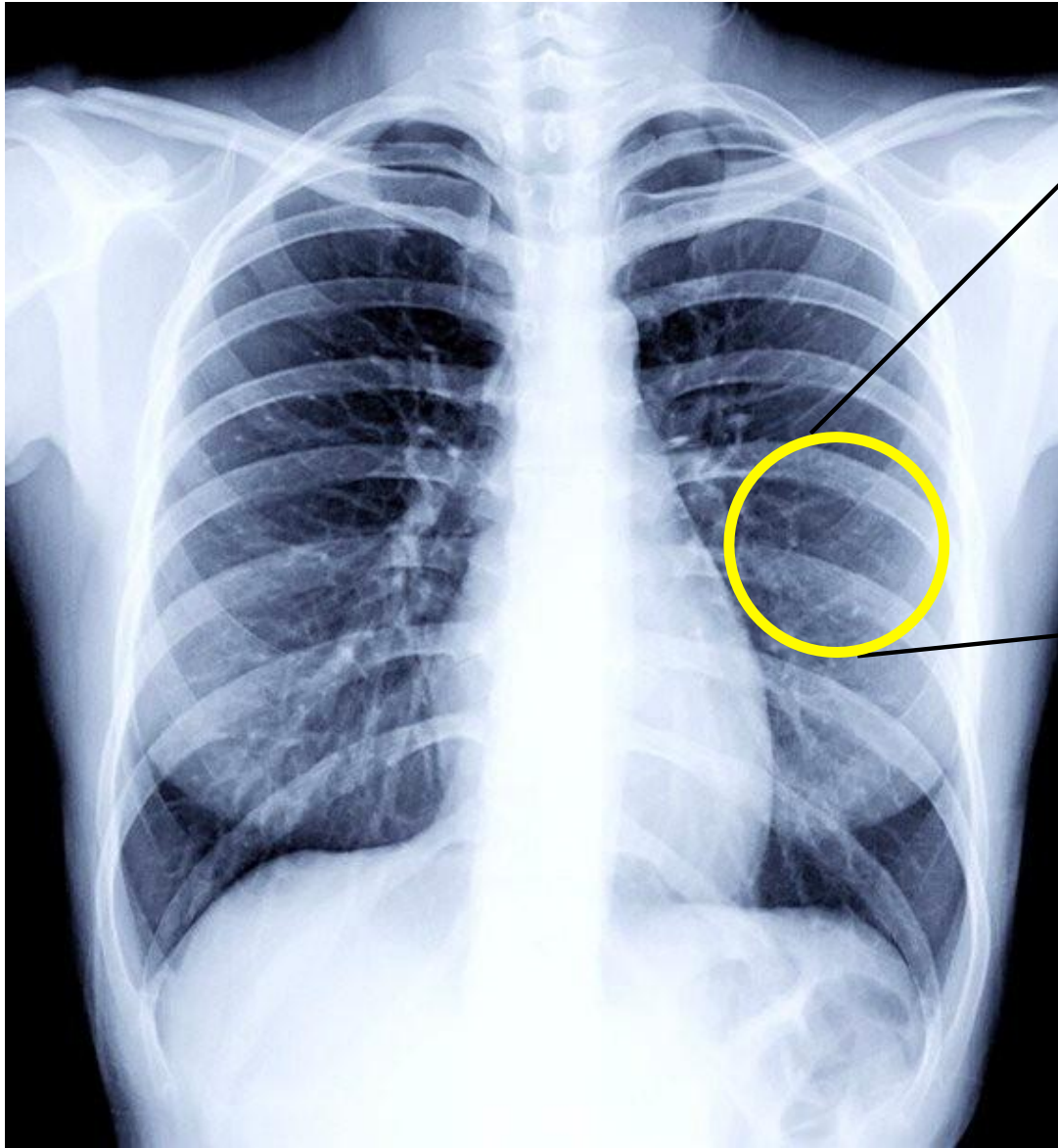
Data Dimensionality:



Data Dimensionality:

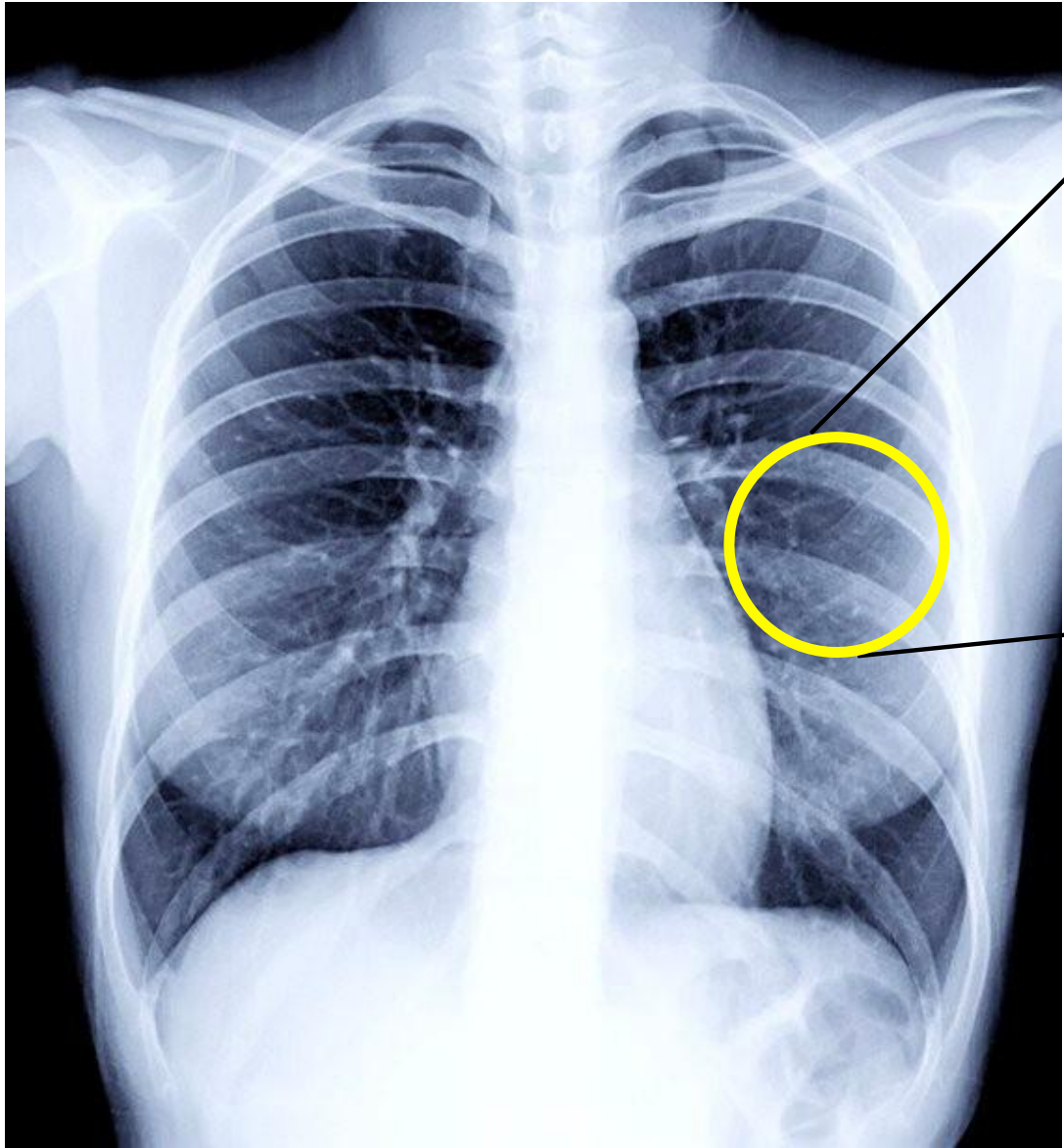


What is a PIXEL:



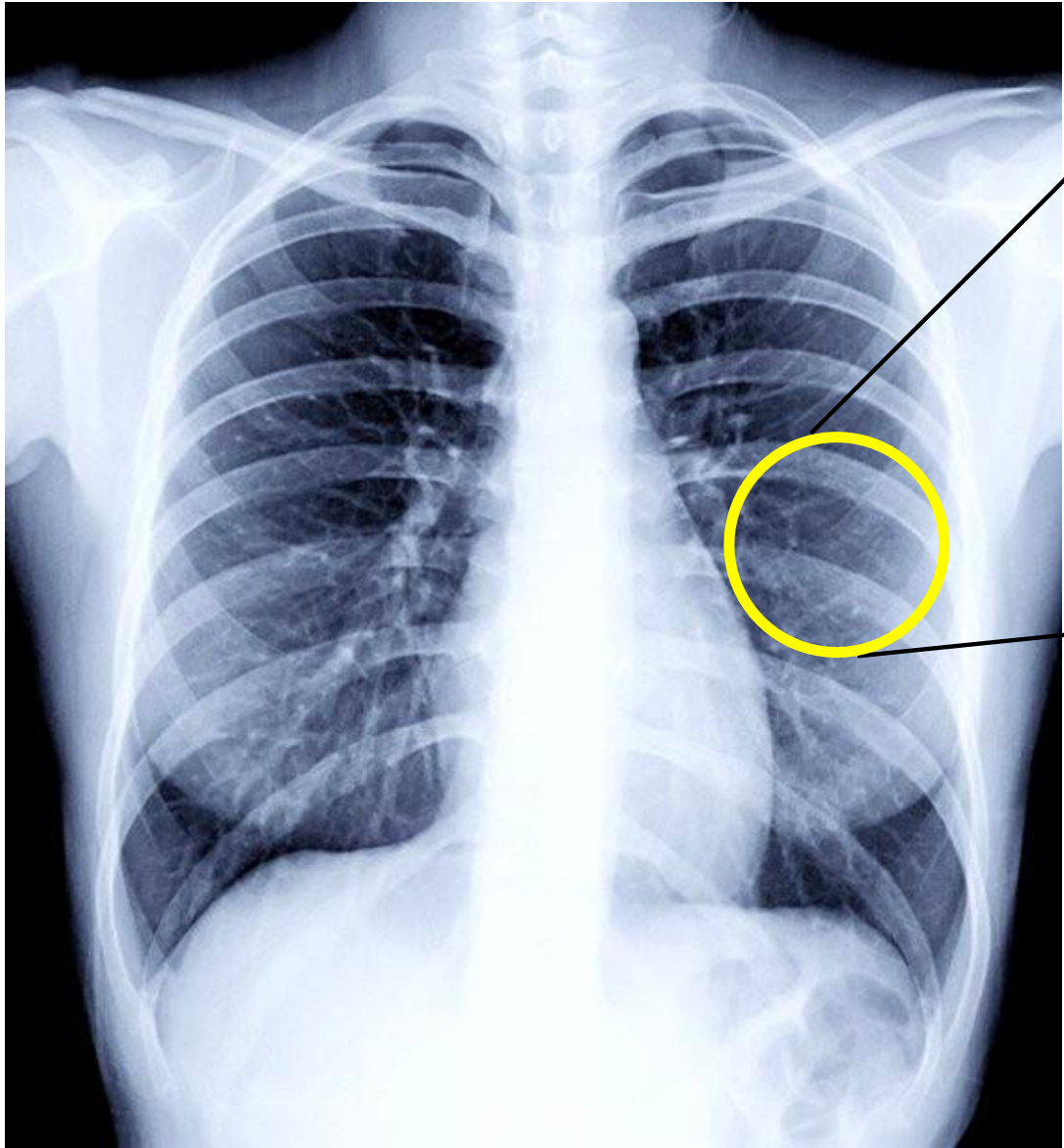
200	187	142	96
0	56	230	117
85	164	22	25
76	149	61	255

What is a PIXEL:



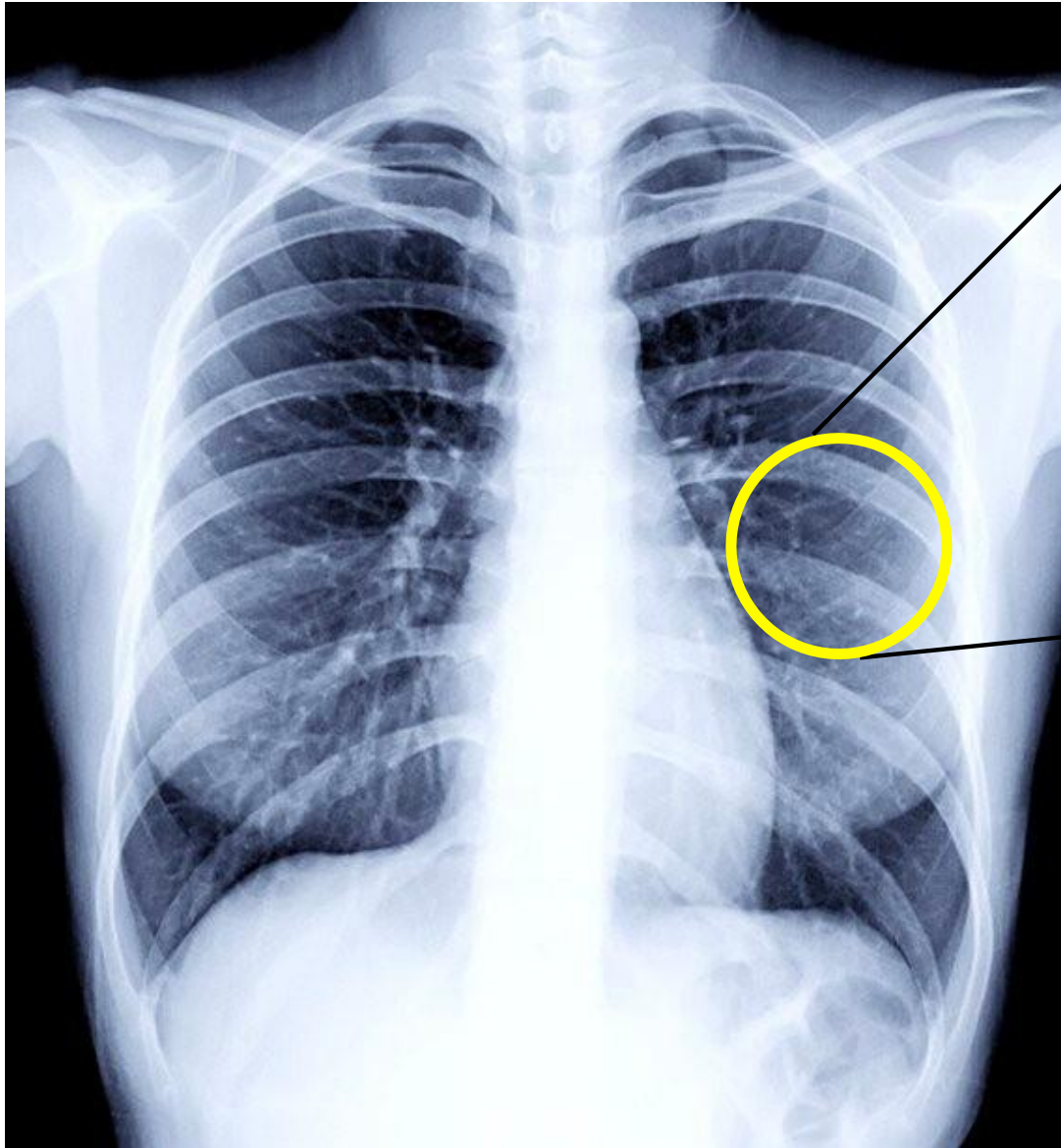
200	187	142	96
0	56	230	117
85	164	22	25
76	149	61	

What is a PIXEL:



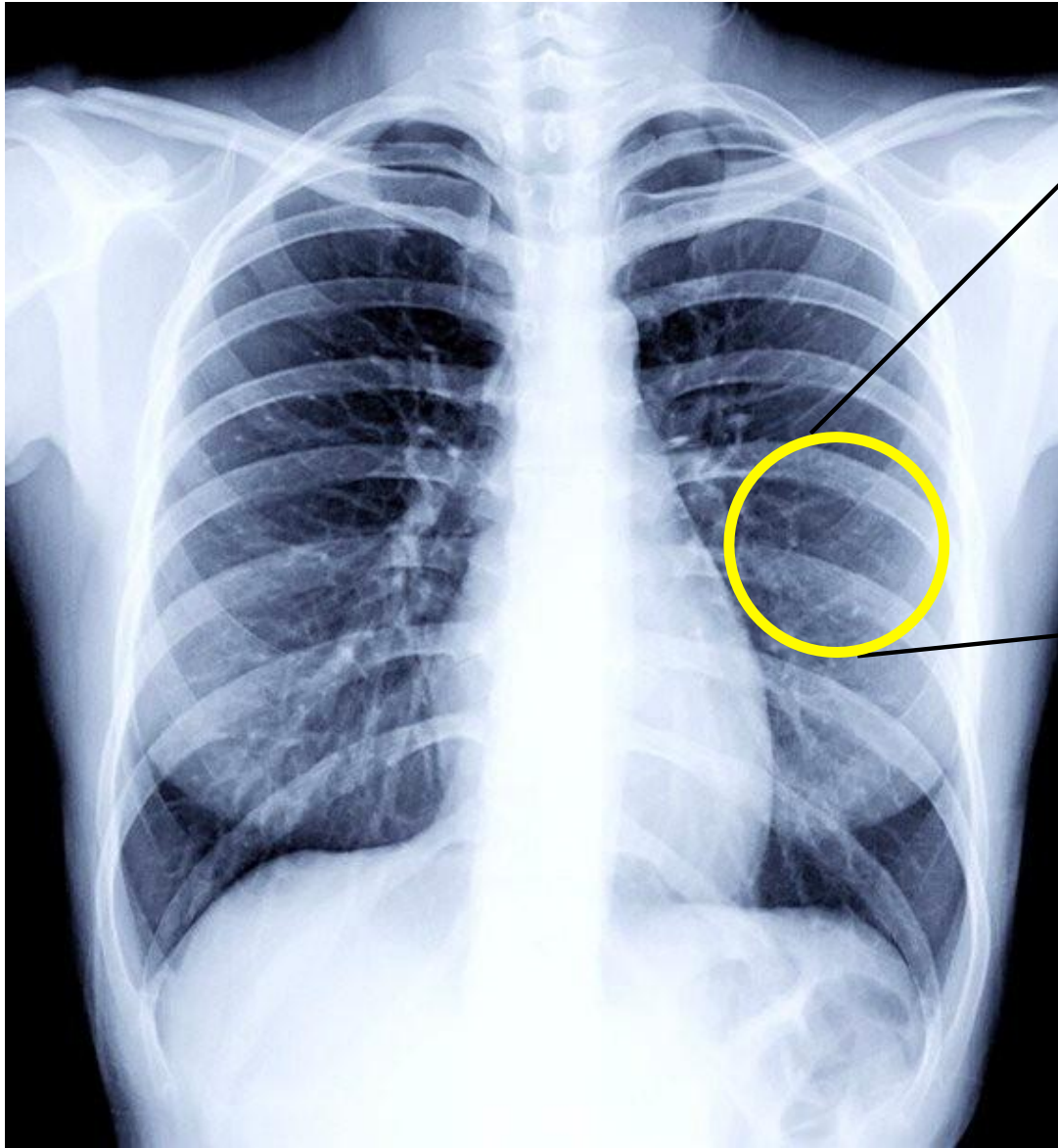
200	187	142	96
	56	230	117
85	164	22	25
76	149	61	

What is a PIXEL:



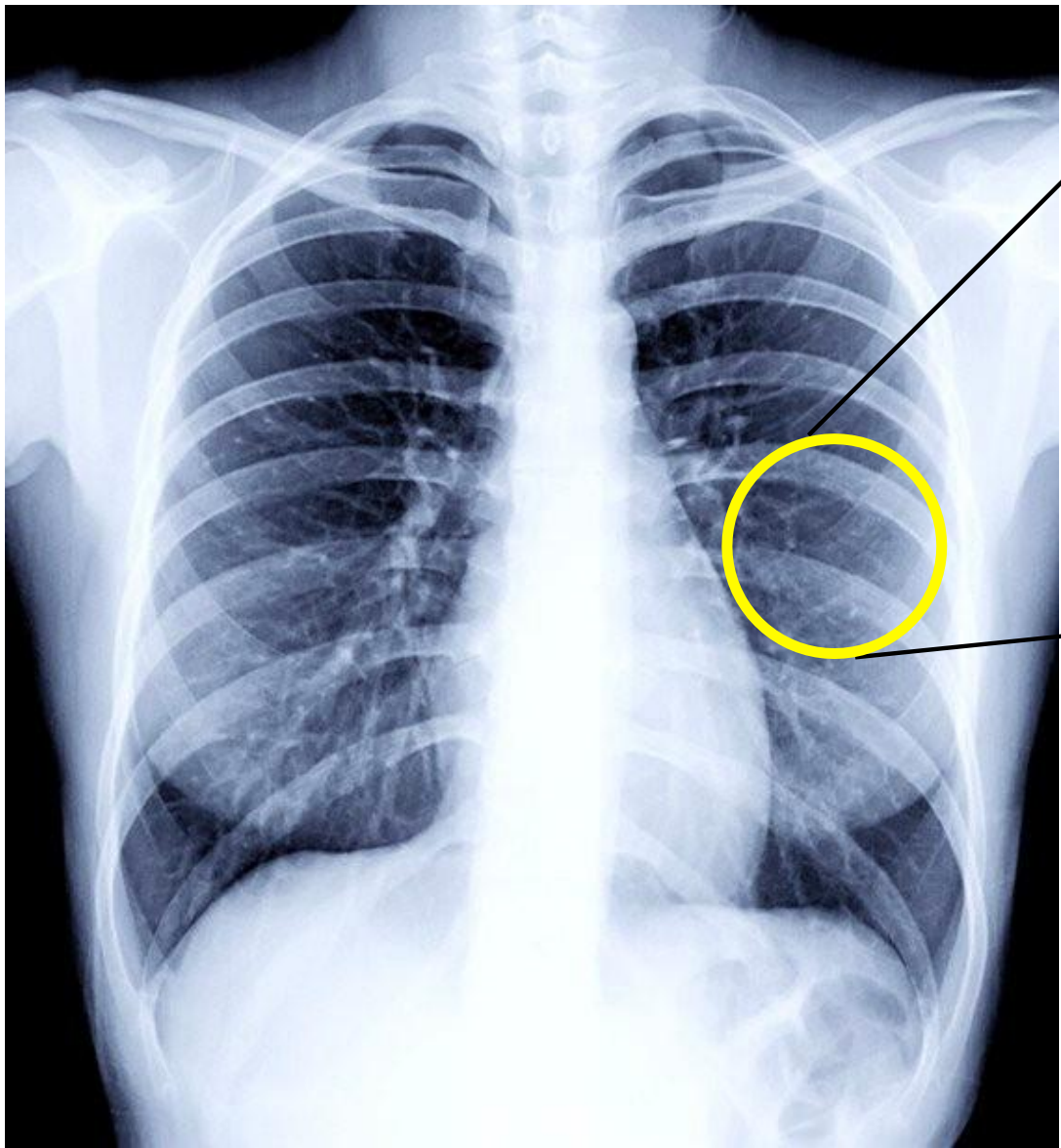
200	187	142	96
	56	230	
85	164	22	25
76	149	61	

What is a PIXEL:



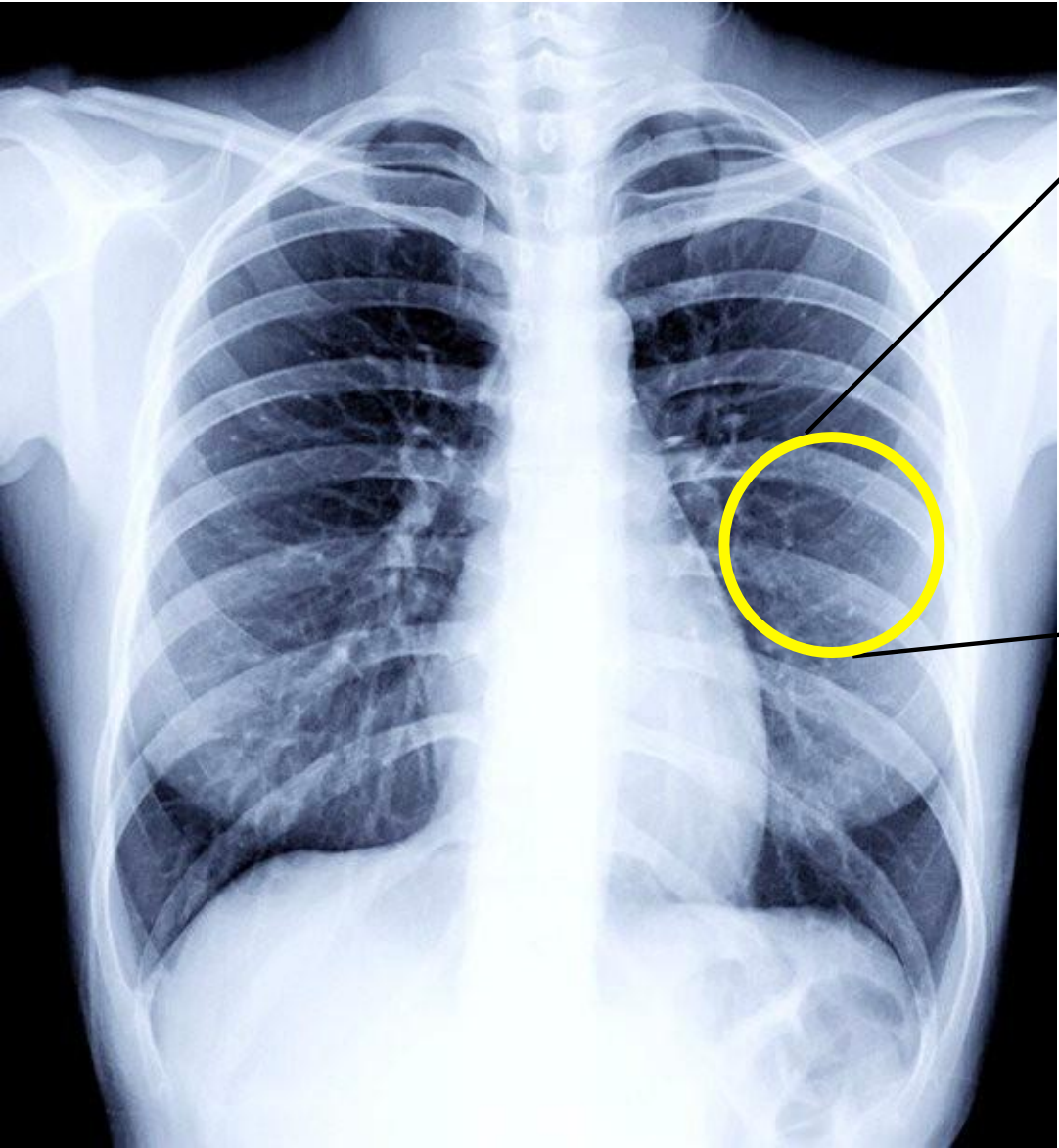
200		142	96
	56	230	
85	164	22	25
76	149	61	

What is a PIXEL:



200		142	96
	56	230	
85	164		25
76	149	61	

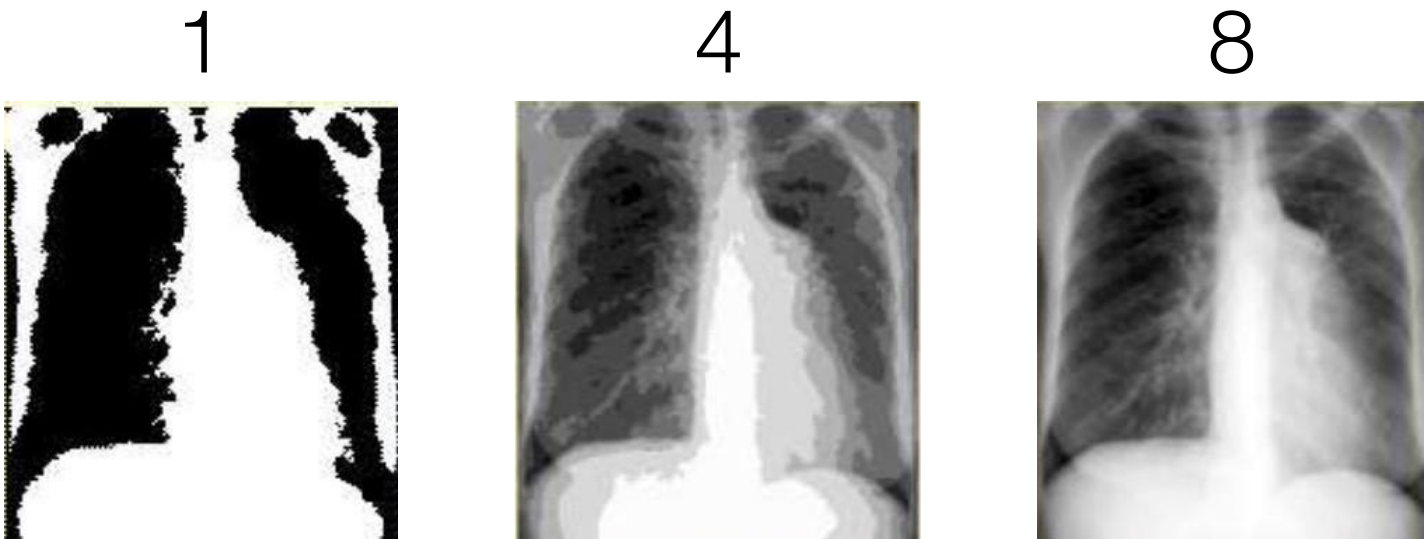
What is a PIXEL:



200		142	96
	56	230	
85	164		25
76	149	61	

Digital Image Pixel Bit Depth

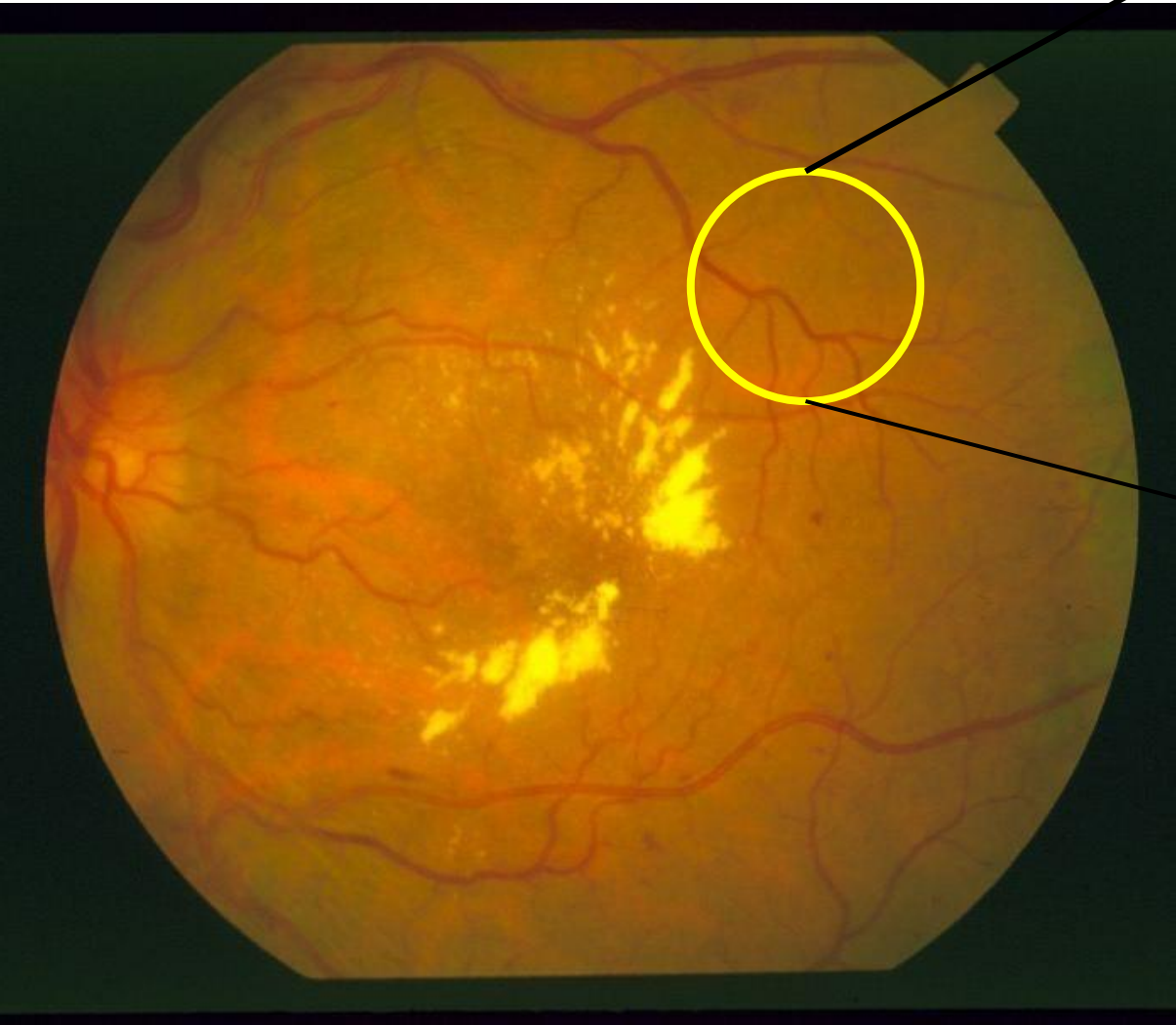
Bits per Pixel



2 16 255

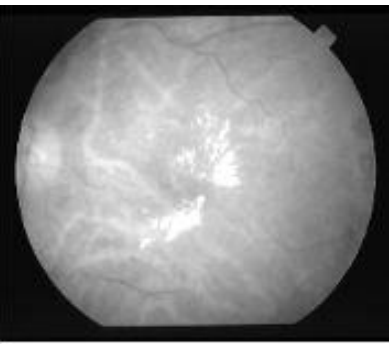
Gray Levels

What is a PIXEL:

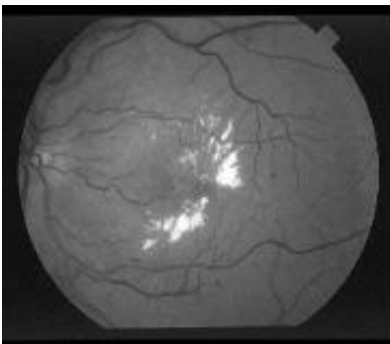


					3
				13	17
200	187	142	96	17	6
0	56	230	117	96	47
85	164	22	25	47	
76	149	61	255		
				R	G
					B

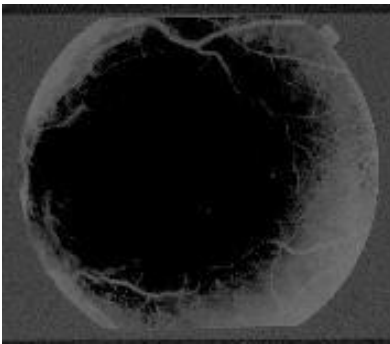
Digital Image Color Channels



R



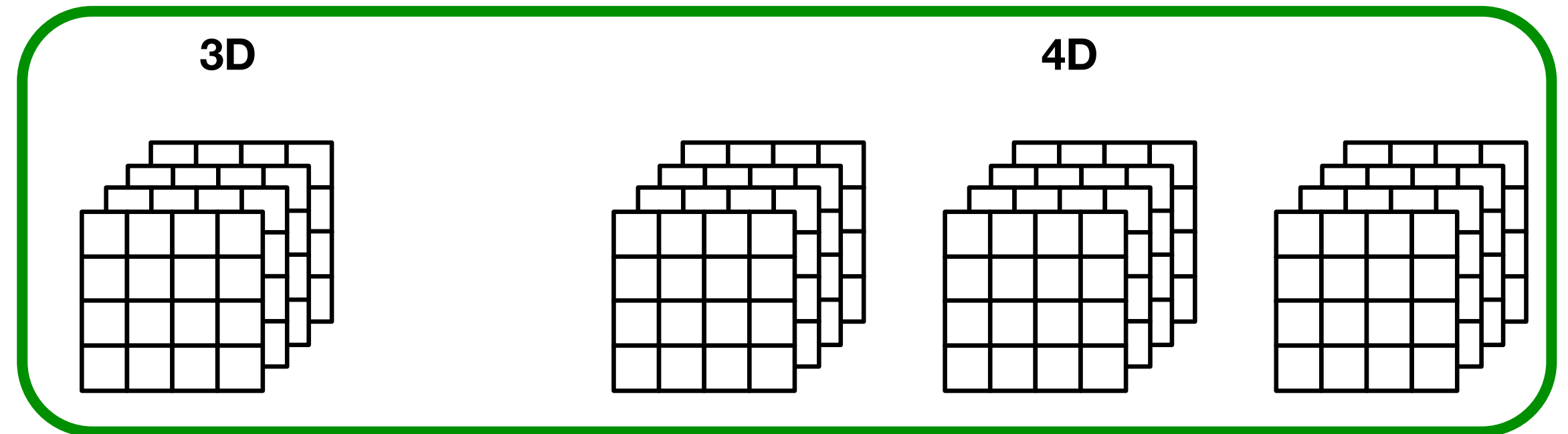
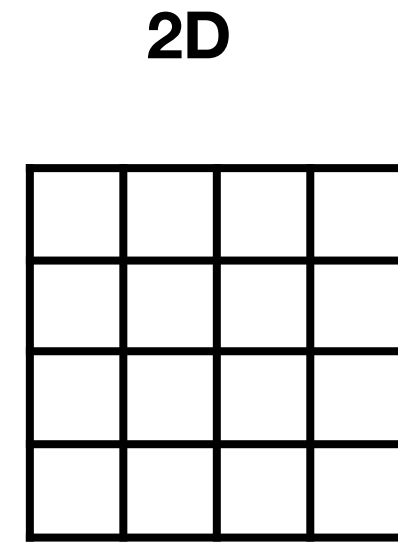
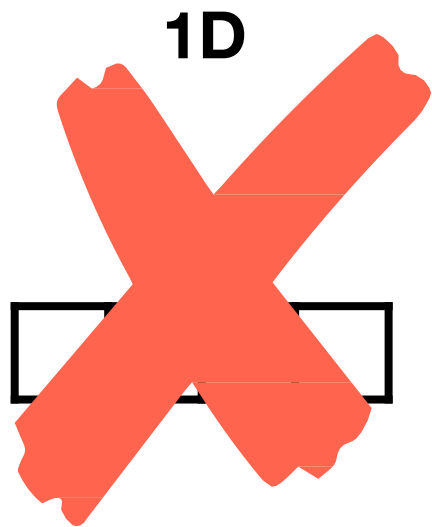
B



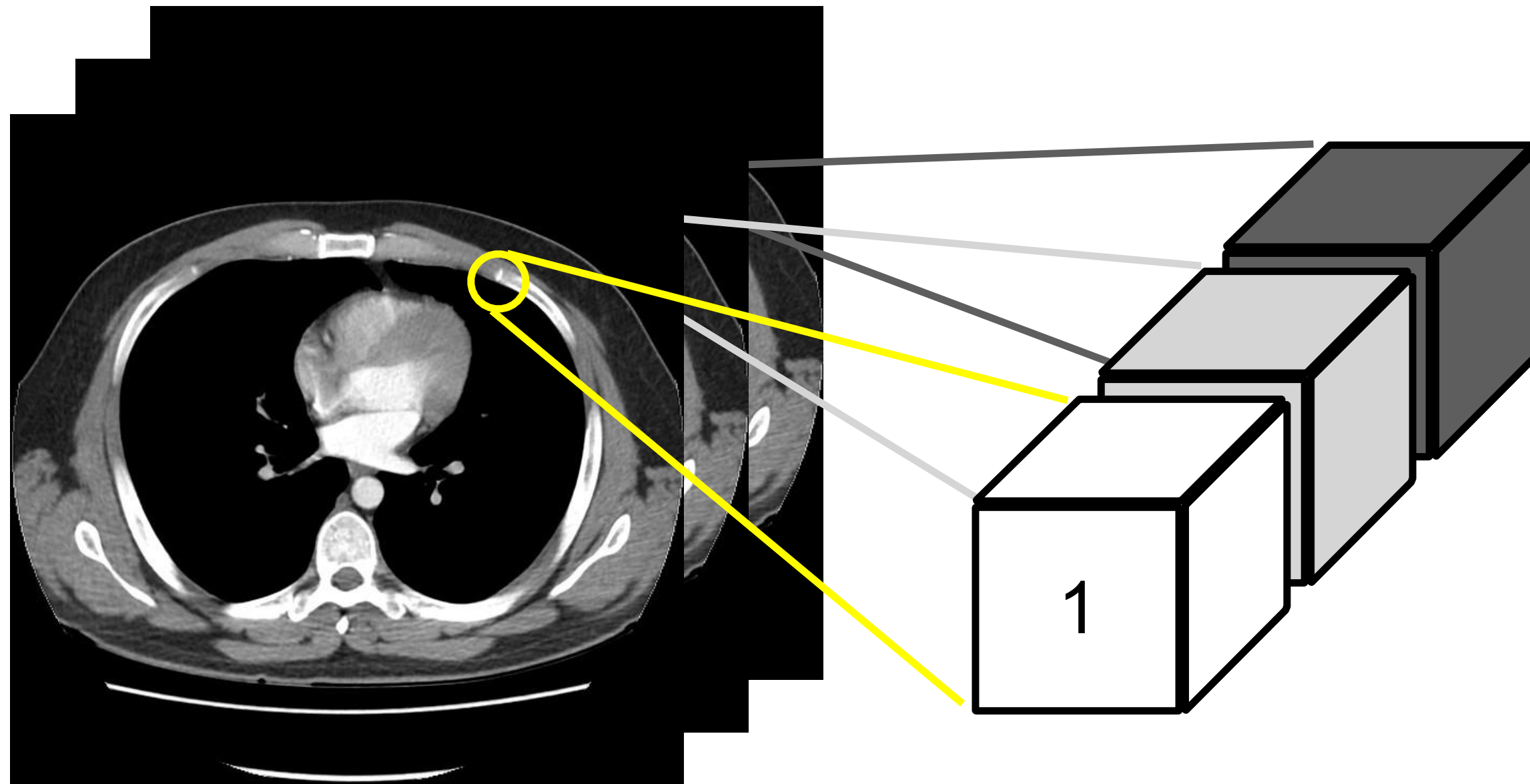
G

Different color channel shows the image content differently !

Data Dimensionality:

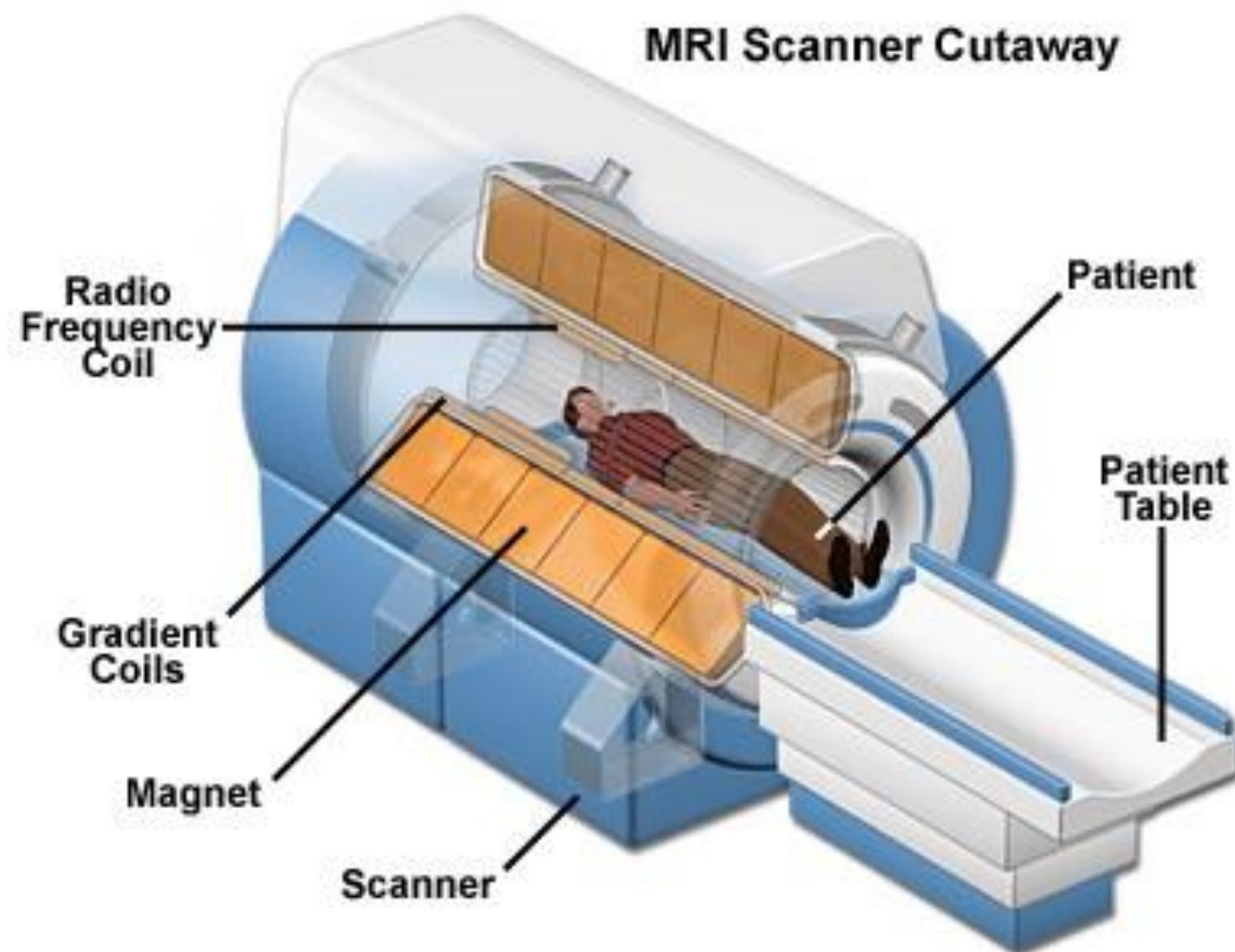


What is a VOXEL:

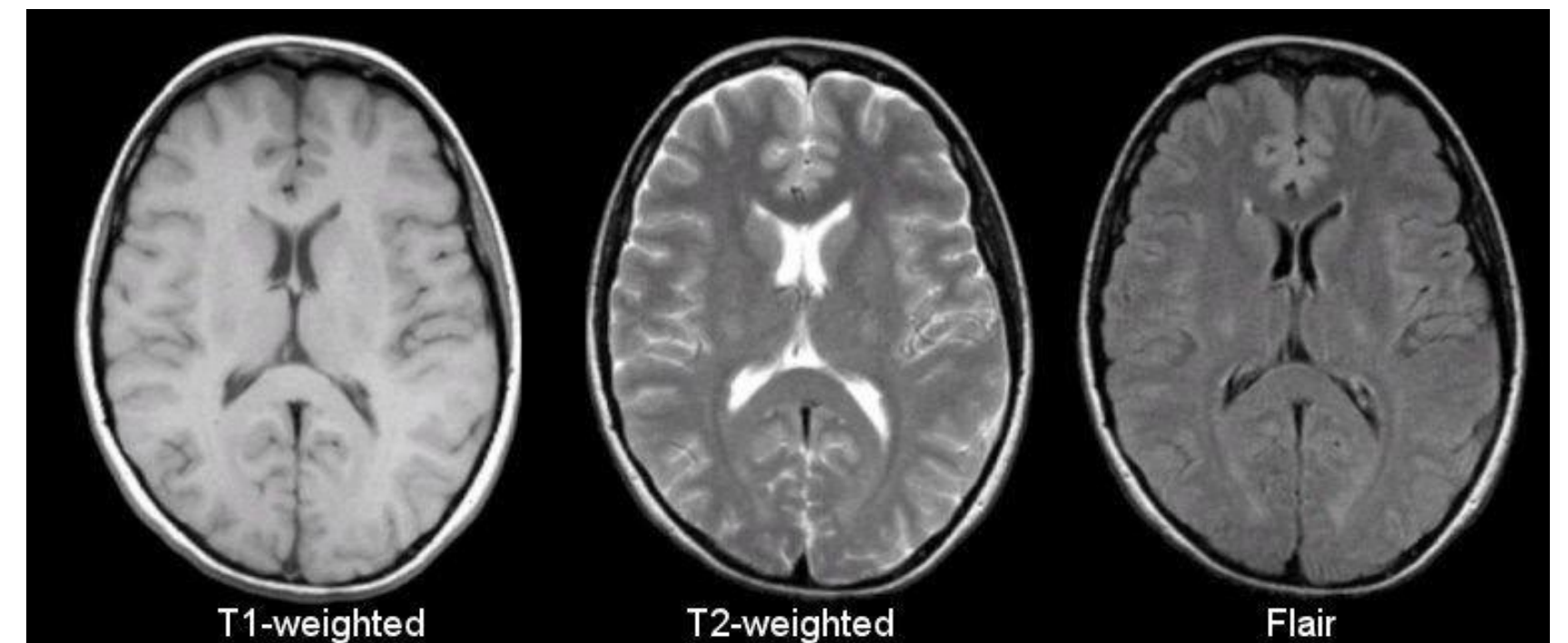


The voxel value represents the mean contribution of different tissues projected on a 2D plane

Magnetic Resonance Imaging (MRI)

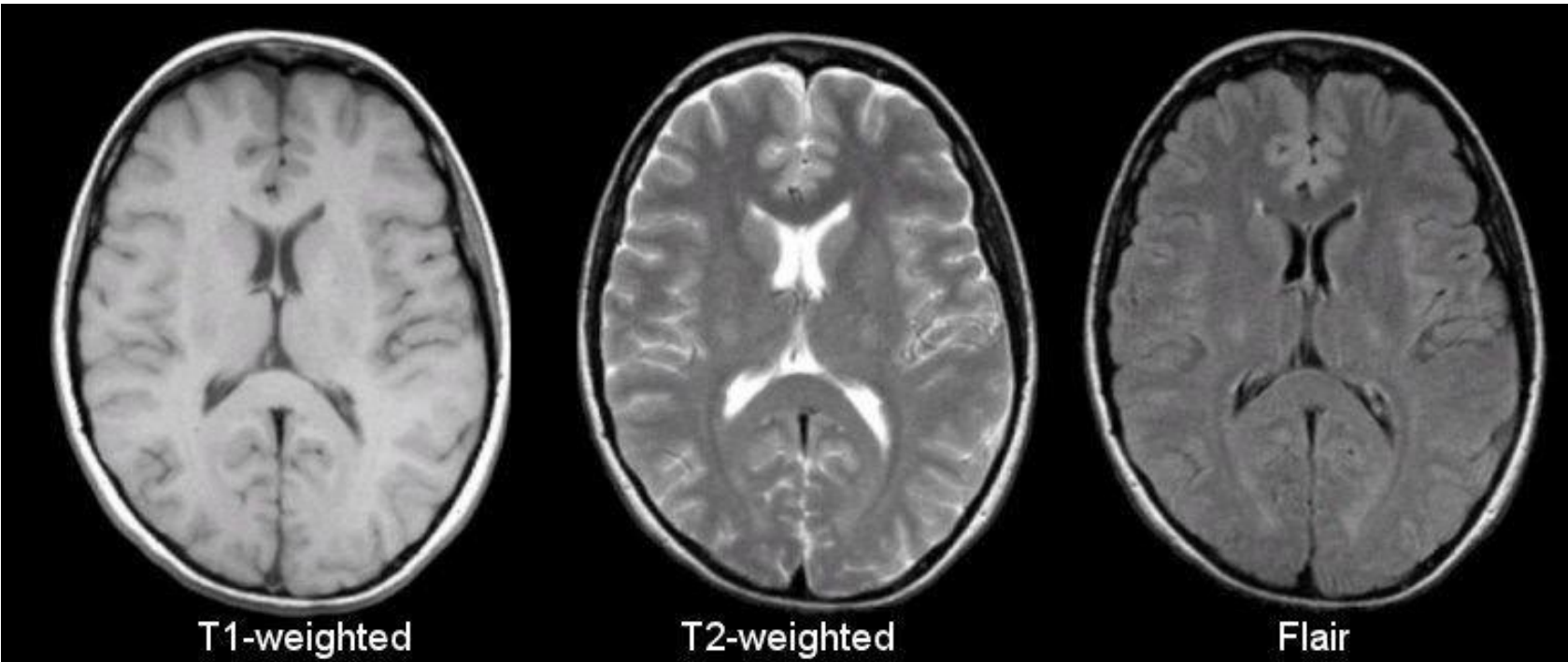


- 3D, Expensive, No Radiation
- Super strong magnet
- Exploits physical properties of the imaged material
- Different acquisition protocols produce image with variable contrasts



<https://youtu.be/jLnuPKhKXVM?si=wVo6NvTRNhnCxPEU>

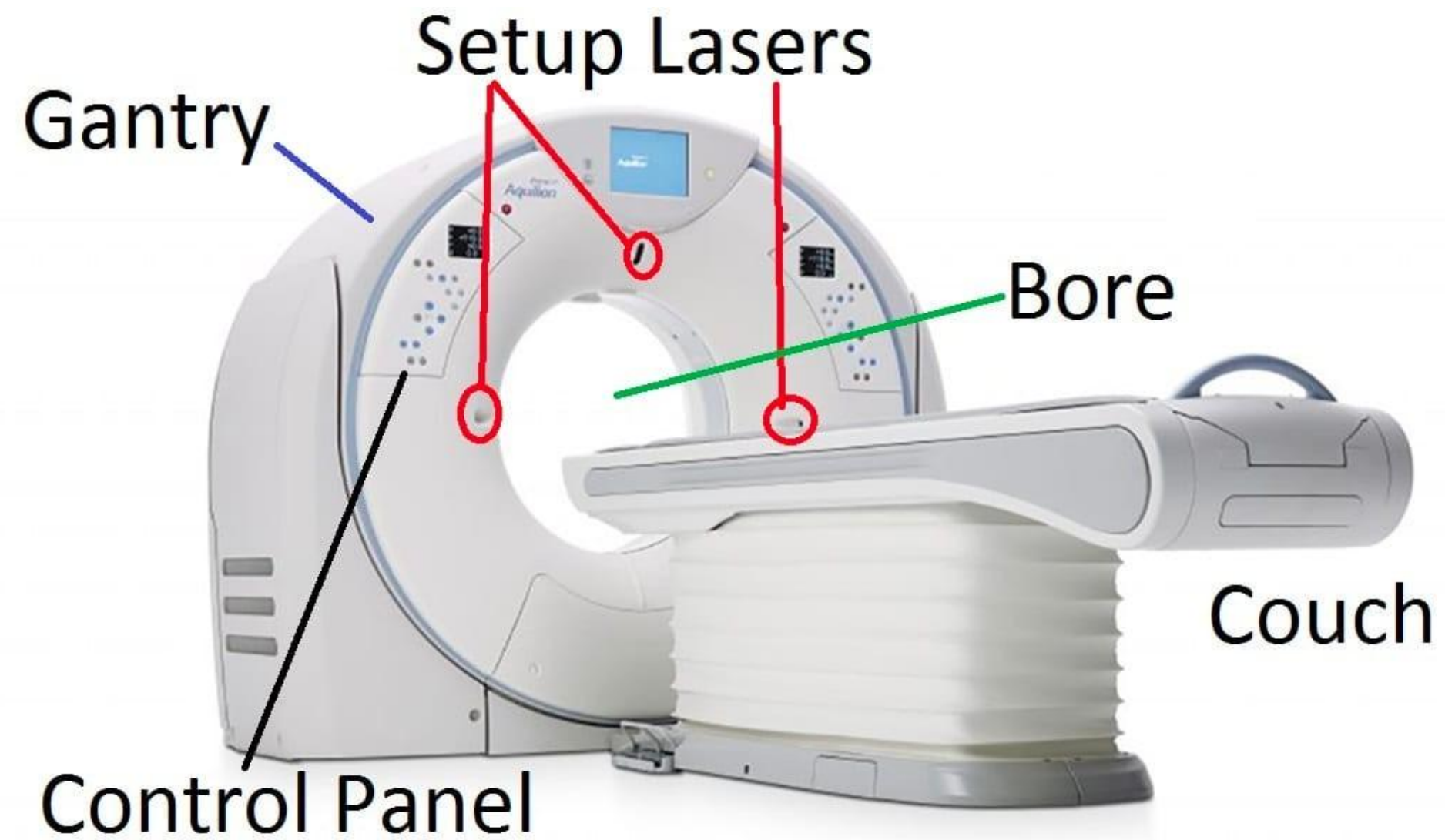
NIfTI - Neuroimaging Informatics Technology Initiative



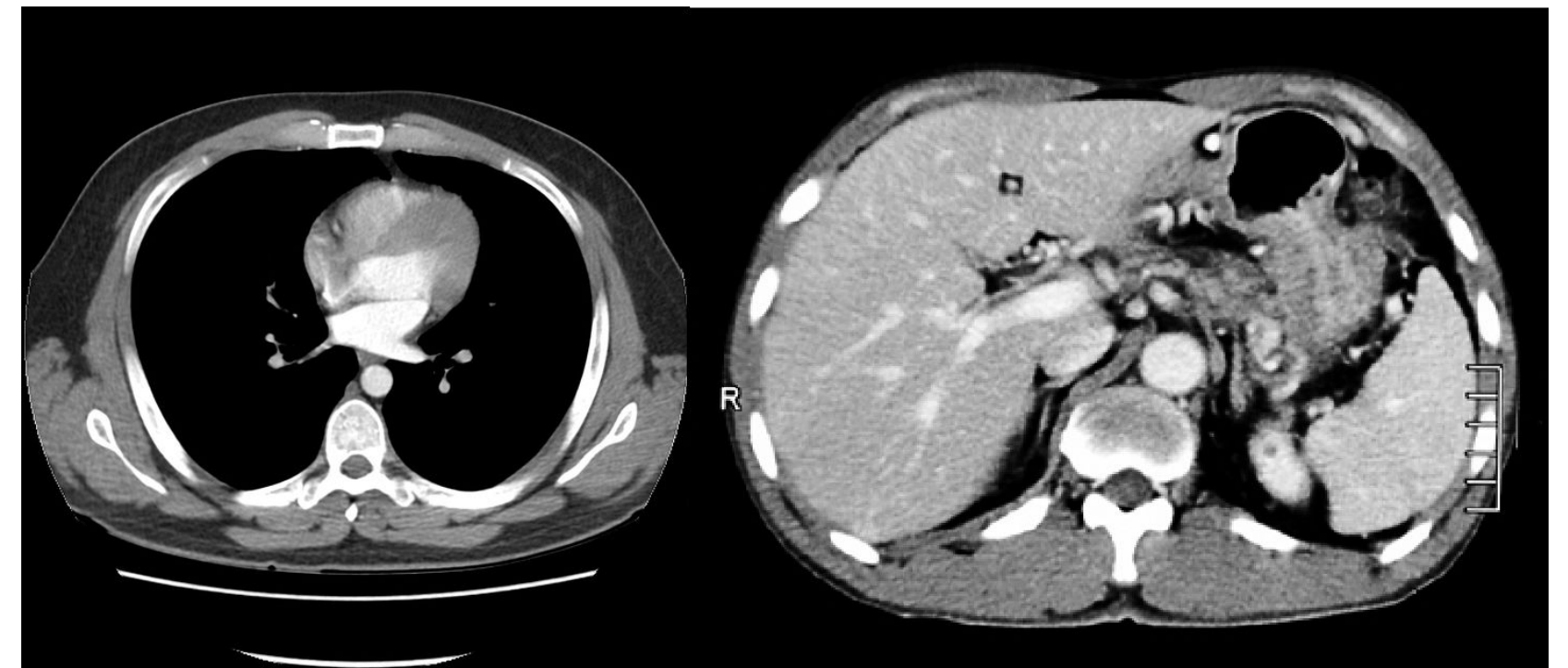
- Primarily employed to store neurological images a.k.a. images of the brain
- Readable by the majority of the medical image viewers (e.g., ITKSnap, 3D Slicer, etc)

Format	Extension	Header	Data Types
NIfTI	.nii	Fixed length 352 byte binary format	Signed and Unsigned int from 8 to 64 bit Float from 32 to 128 bit

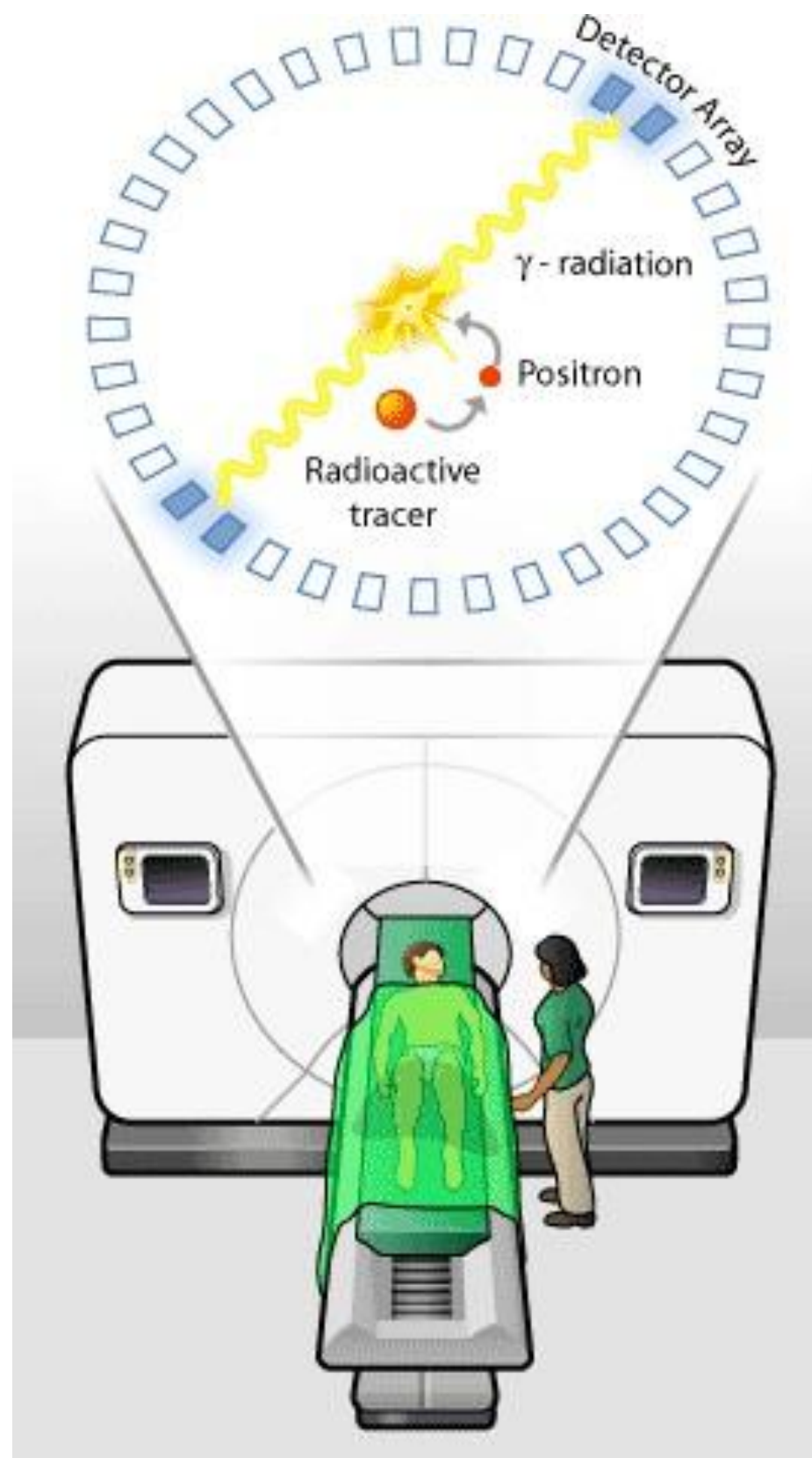
Computed Tomography (CT)



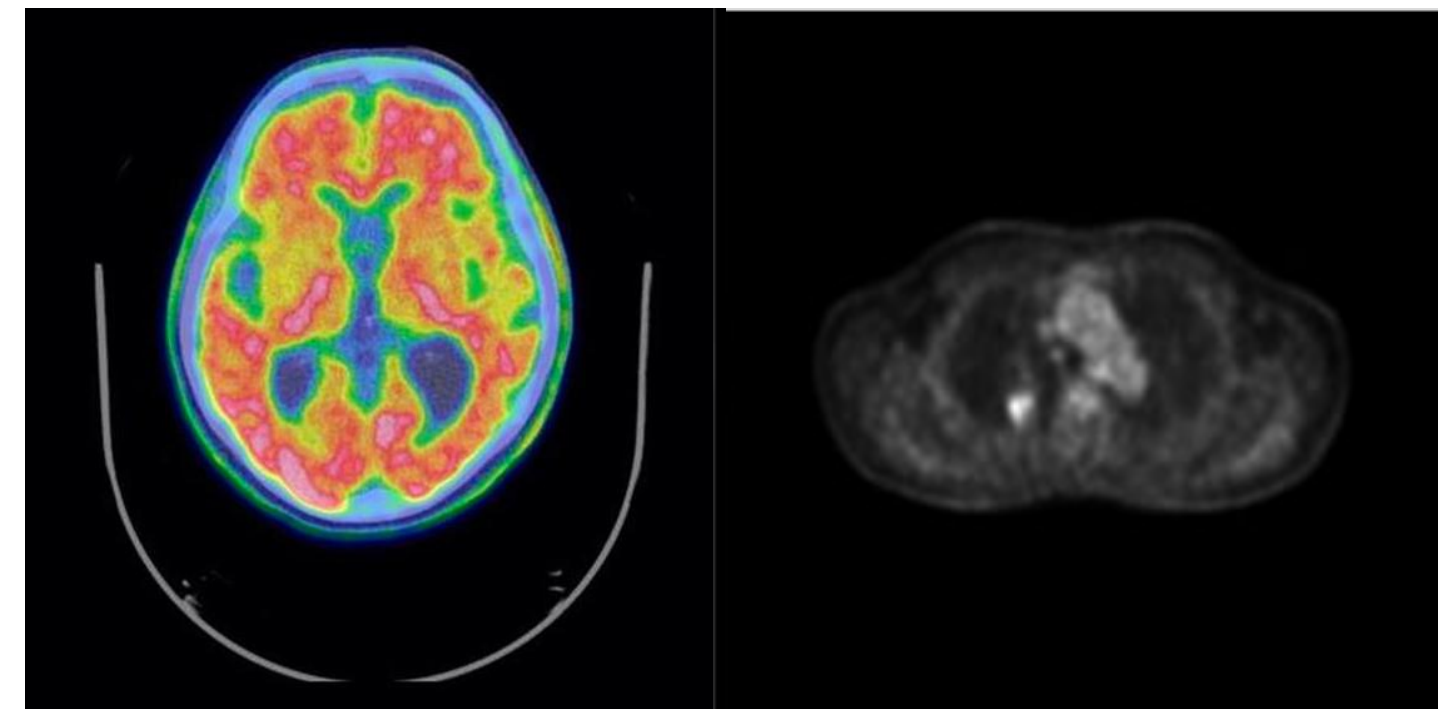
- High 3D Resolution, Expensive, With Radiation, Structural
- Capturing the shadow in a spiral line
- Exploits density properties of the imaged material



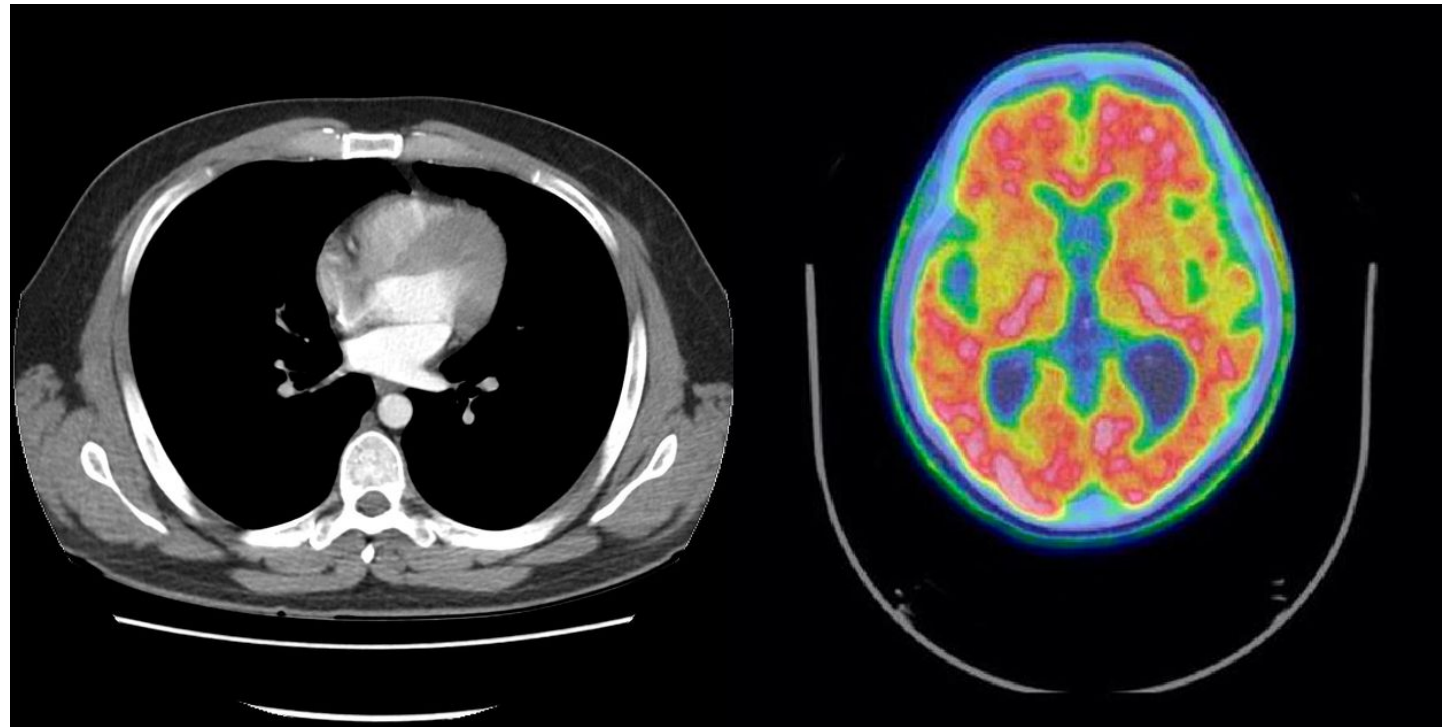
Positron Emission Tomography (PET)



- Measures metabolic activity, with Radiation, Expensive, Functional
- Can be acquire coupled with the CT
- Exploits density properties of the imaged material



DICOM - Digital Imaging and COmmunication in Medicine



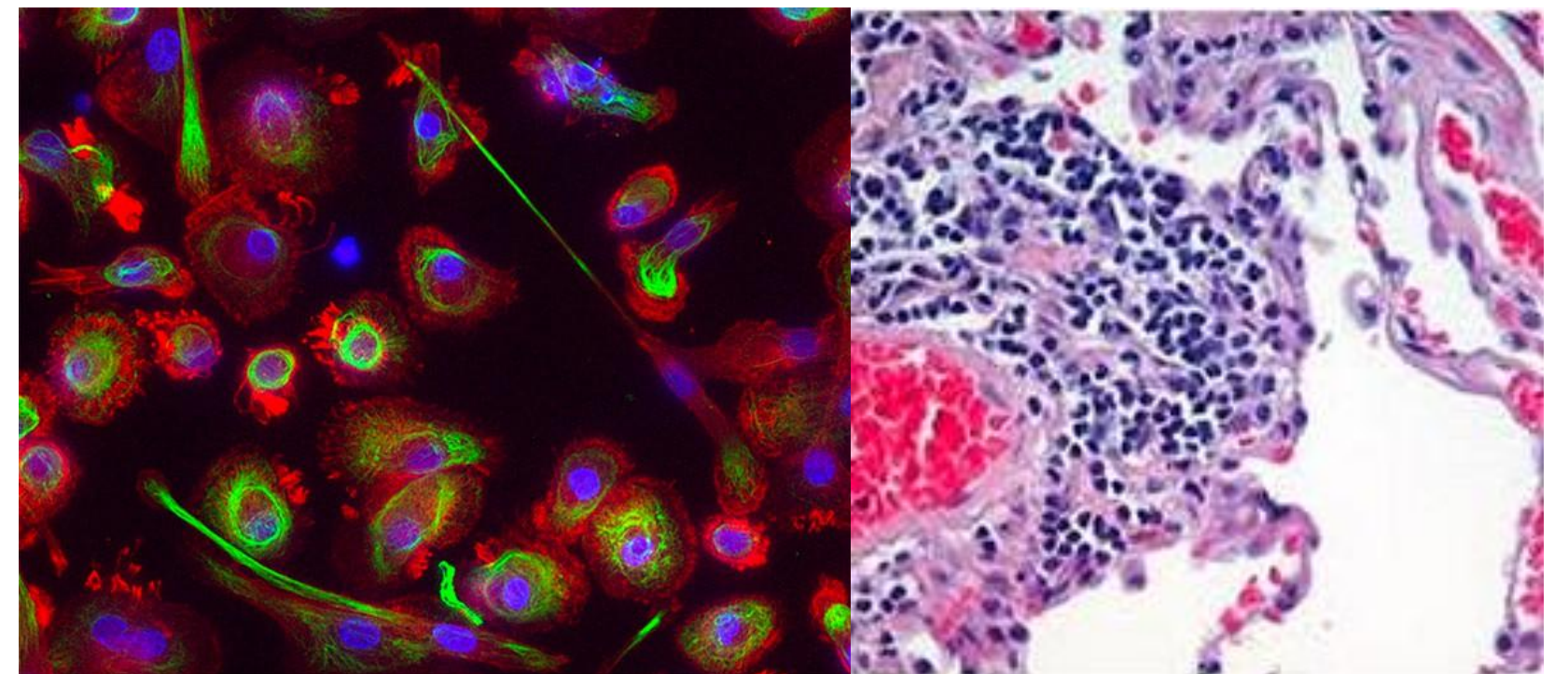
- Primarily employed to store CT and PET images
- Readable by the majority of the medical image viewers (e.g., ITKSnap, 3D Slicer, etc)

Format	Extension	Header	Data Types
DICOM	.dcm	Variable length binary format	Signed and Unsigned int from 8 to 16 bit

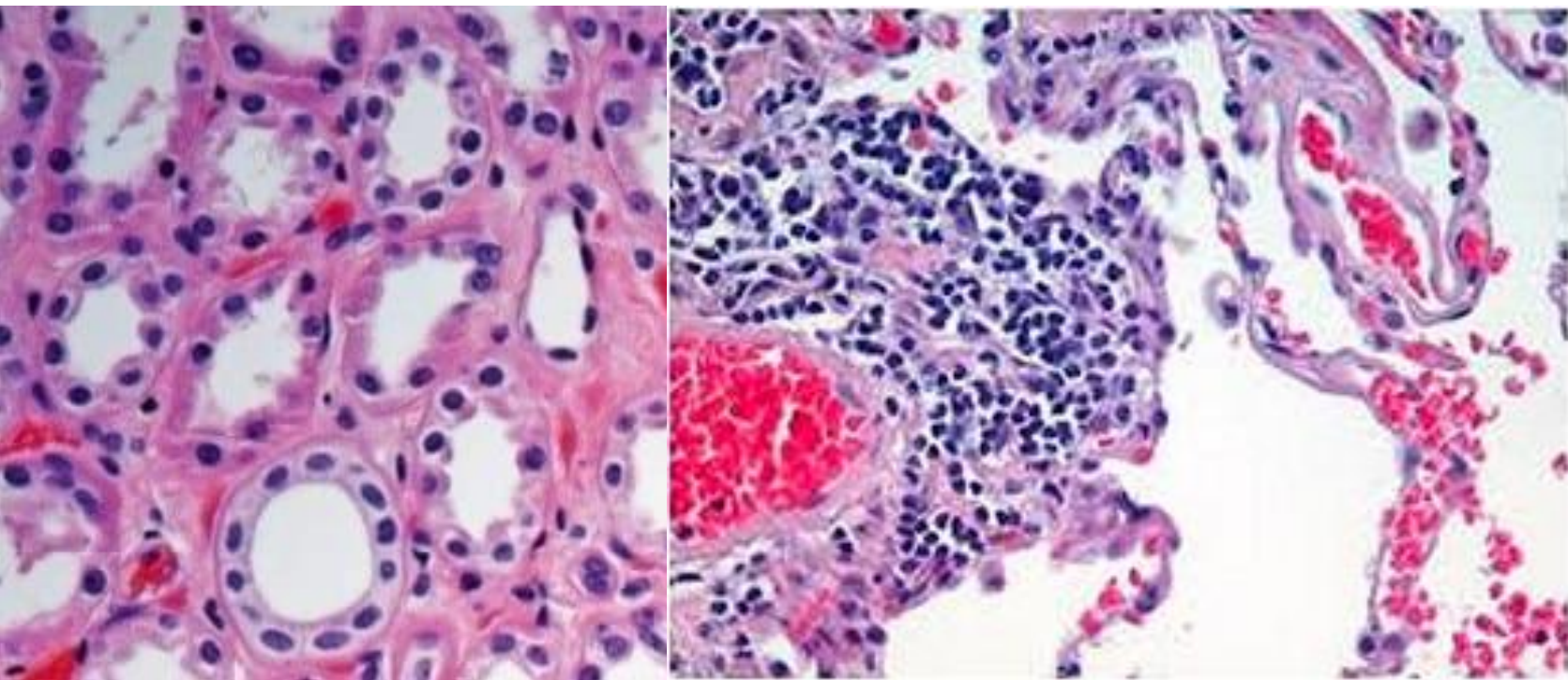
Microscopy(CT)



- Relatively cheap, 2D images, setting up specimen is difficult
- Capturing small tissue sample into super high resolution images
- Exploits different light sources to acquire the images



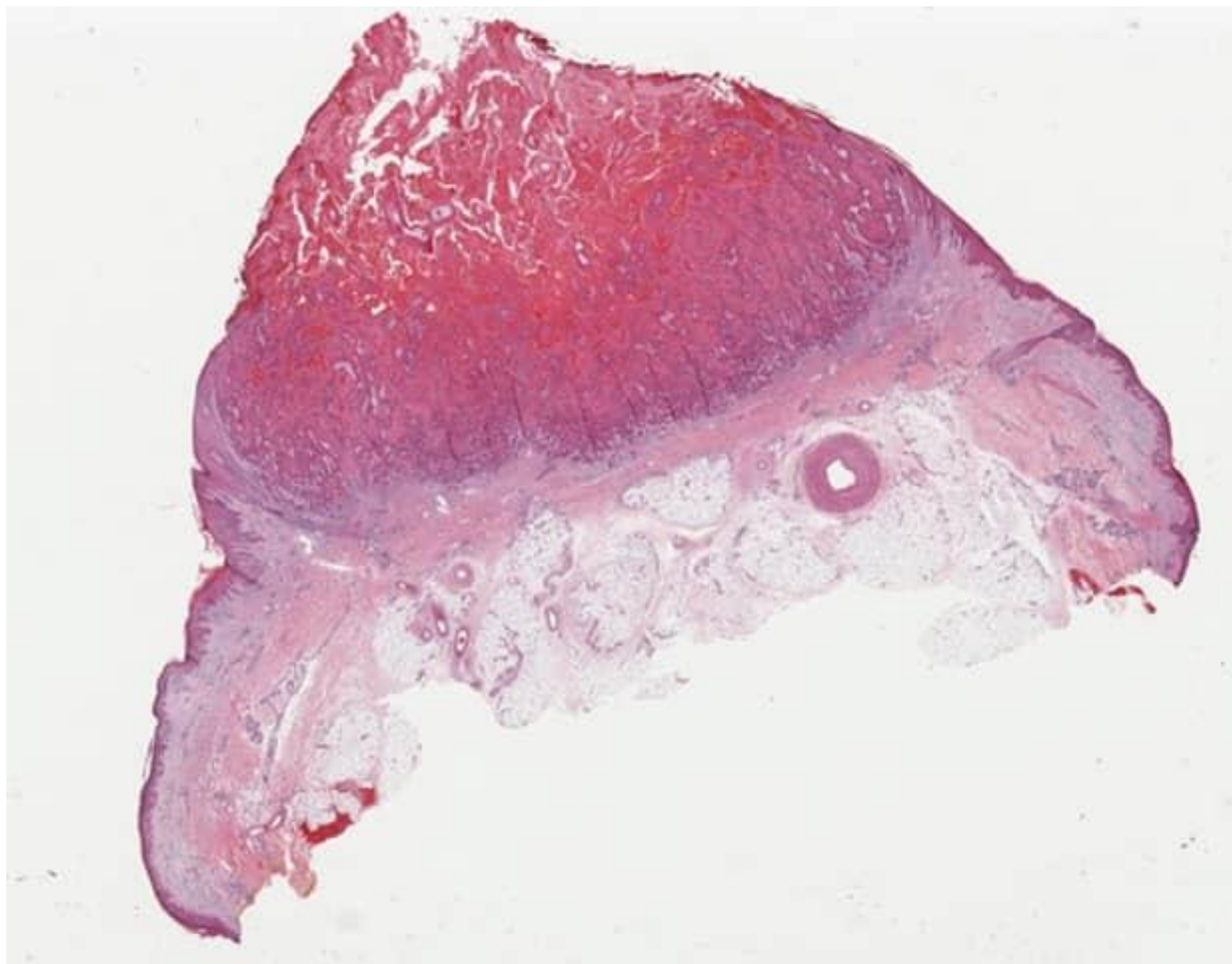
TIFF - Tag Image FileFormat



- Primarily employed to store 2D images (e.g., histopathology)
- Readable by the majority of the medical image viewers (e.g., ITKSnap, 3D Slicer, etc)

Format	Extension	Header	Data Types
TIFF	.tiff	N.A.	Signed and Unsigned int 8

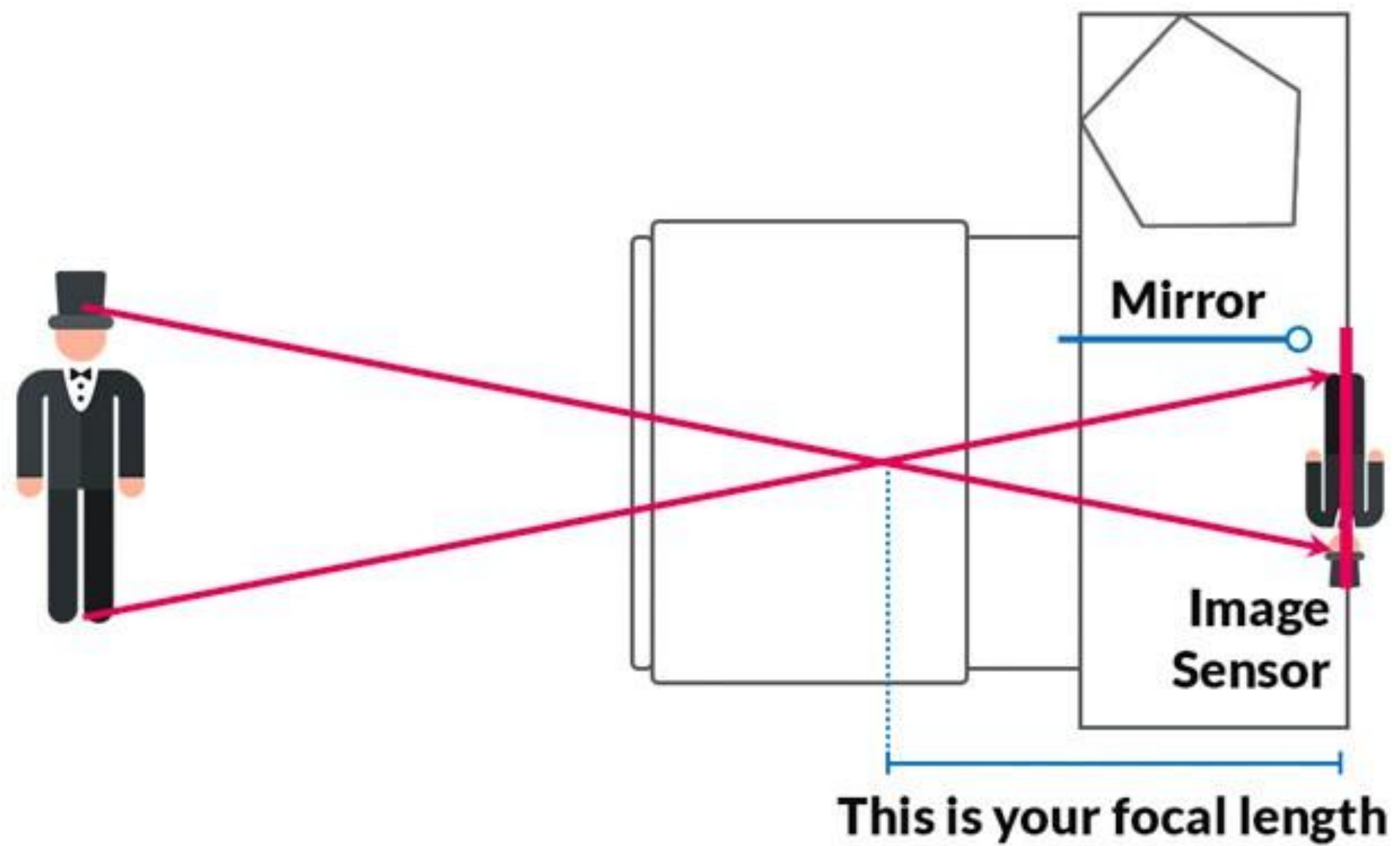
MIRAX - Microscopic Image Repository and eXchange



- Primarily employed to store high-resolution 2D images from digital pathology (e.g. whole slide scanning)
- Primarily used by 3d HISTECH Pannoramic Scanners for high-resolution microscopic tissue slides
- Readable by specialized medical image viewers (e.g. QuPath)

Format	Extension	Header	Data Types
MIRAX	.mrxs	Metadata file (.mrxs) + image data directory	8-bit RGB, Grayscale

Camera



- Endless acquirable images
- Captures visible light focused through glass
- Exploits light to acquire the images



PNG and JPEG - Portable Network Graphics and Joint Photographic Experts Group



- Primarily employed to store natural 2D RGB images
- Readable by every image viewers

Format	Extension	Header	Data Types
PNG	.png	N.A.	All
JPEG	.jpeg or .jpg	N.A.	All

Dataset Issues

When dealing with a dataset we can find the following issues:

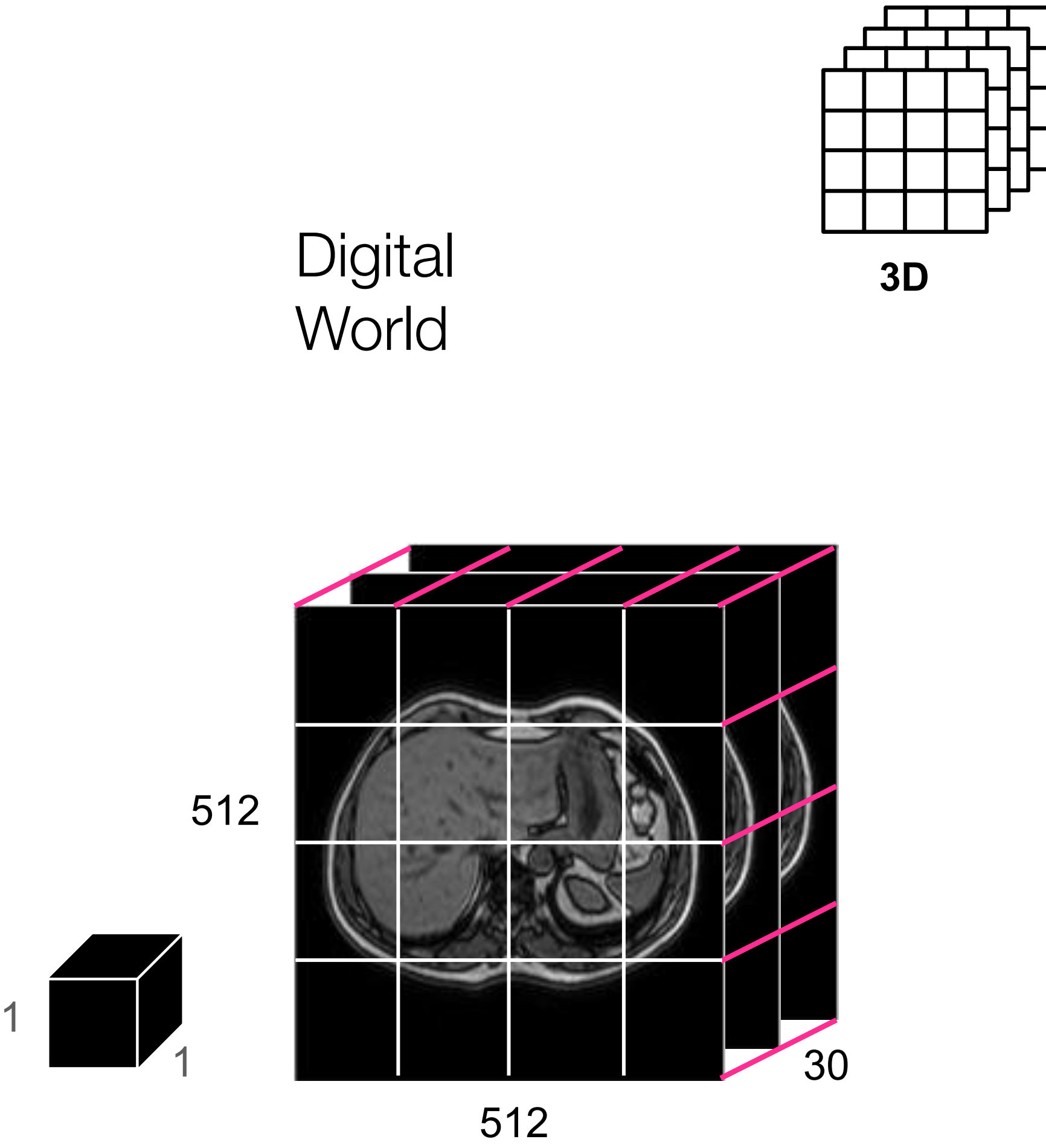
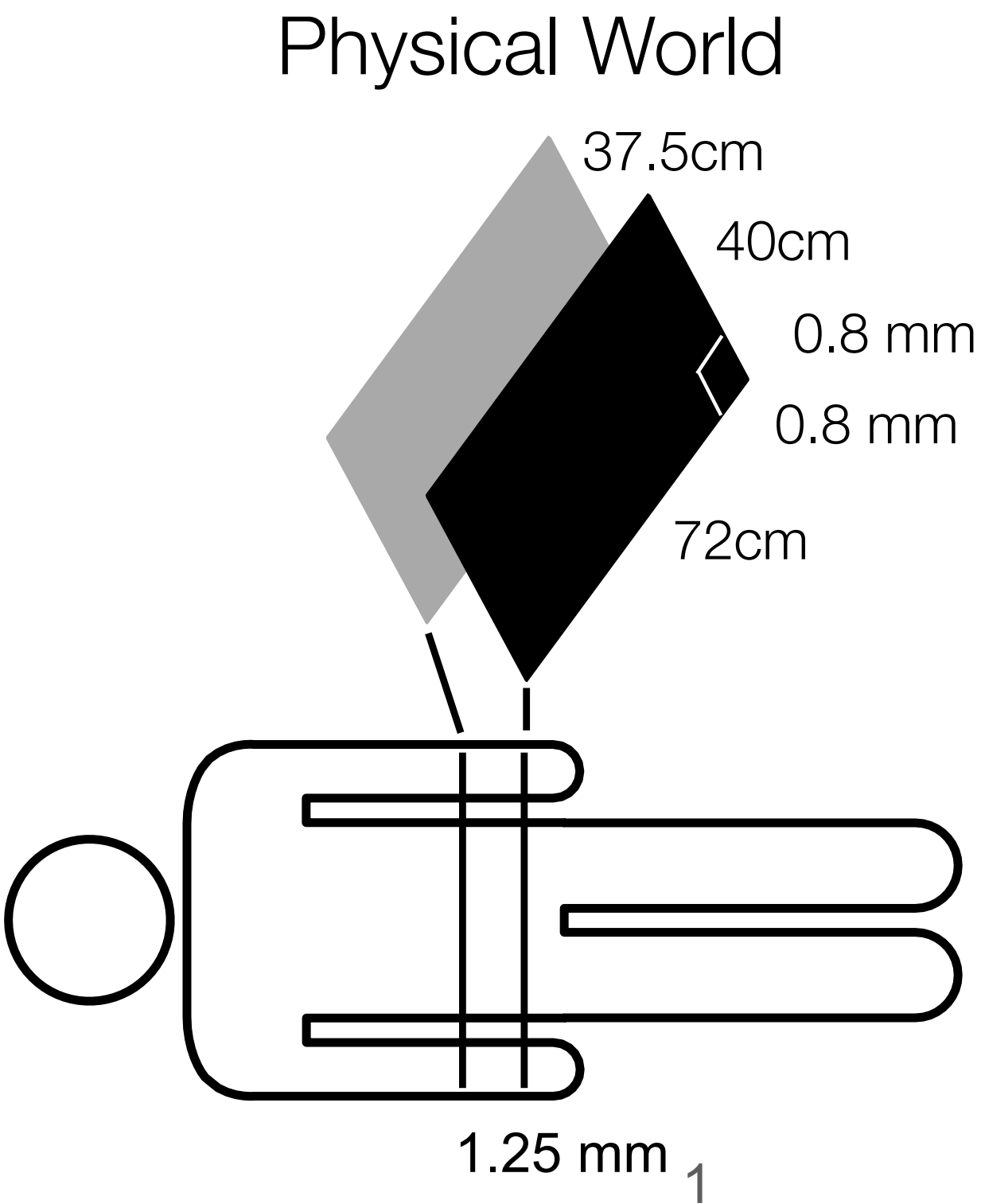
- Different pixel sizes, pixel spacing (a.k.a. slice thickness)
- Different image dimensions
- Variable contrasts
- Variable intensities
- Noise
- Artifacts

Dataset Issues

When dealing with a dataset we can find the following issues:

- Different pixel sizes, pixel spacing (a.k.a. slice thickness)
- Different image dimensions
- Variable contrasts
- Variable intensities
- Noise
- Artifacts

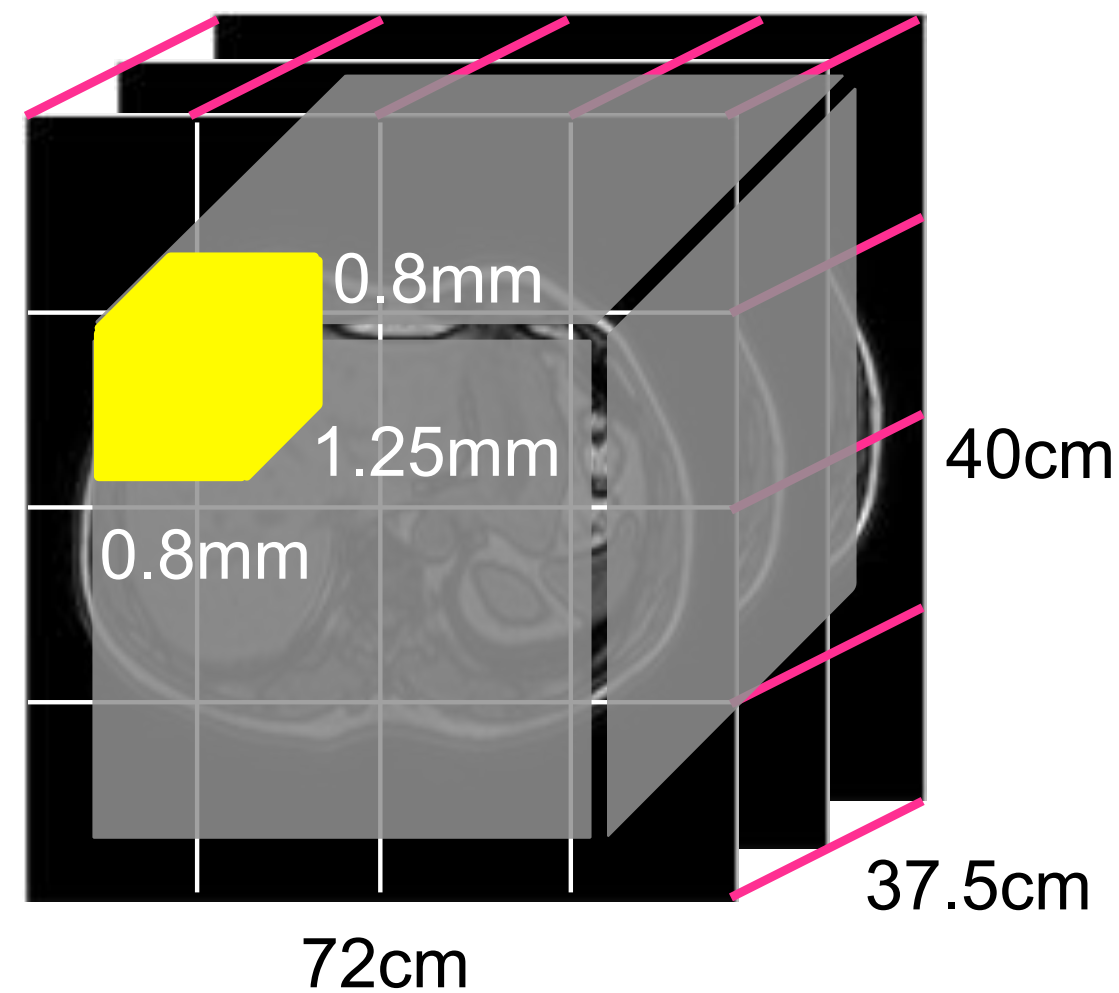
Dimensions mismatch



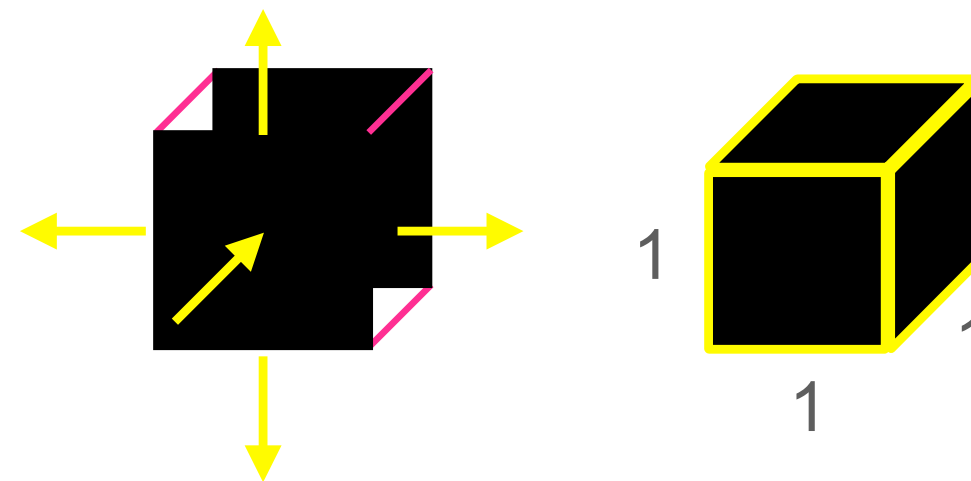
The digital sampling does not match the physical one

Resampling

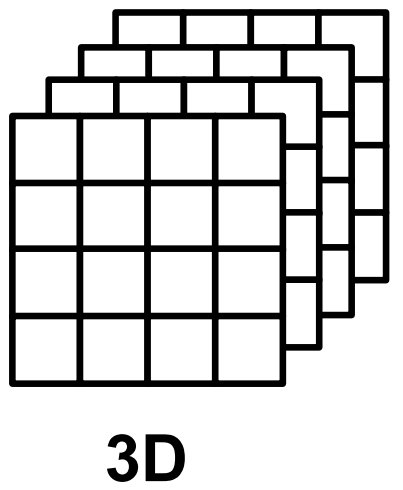
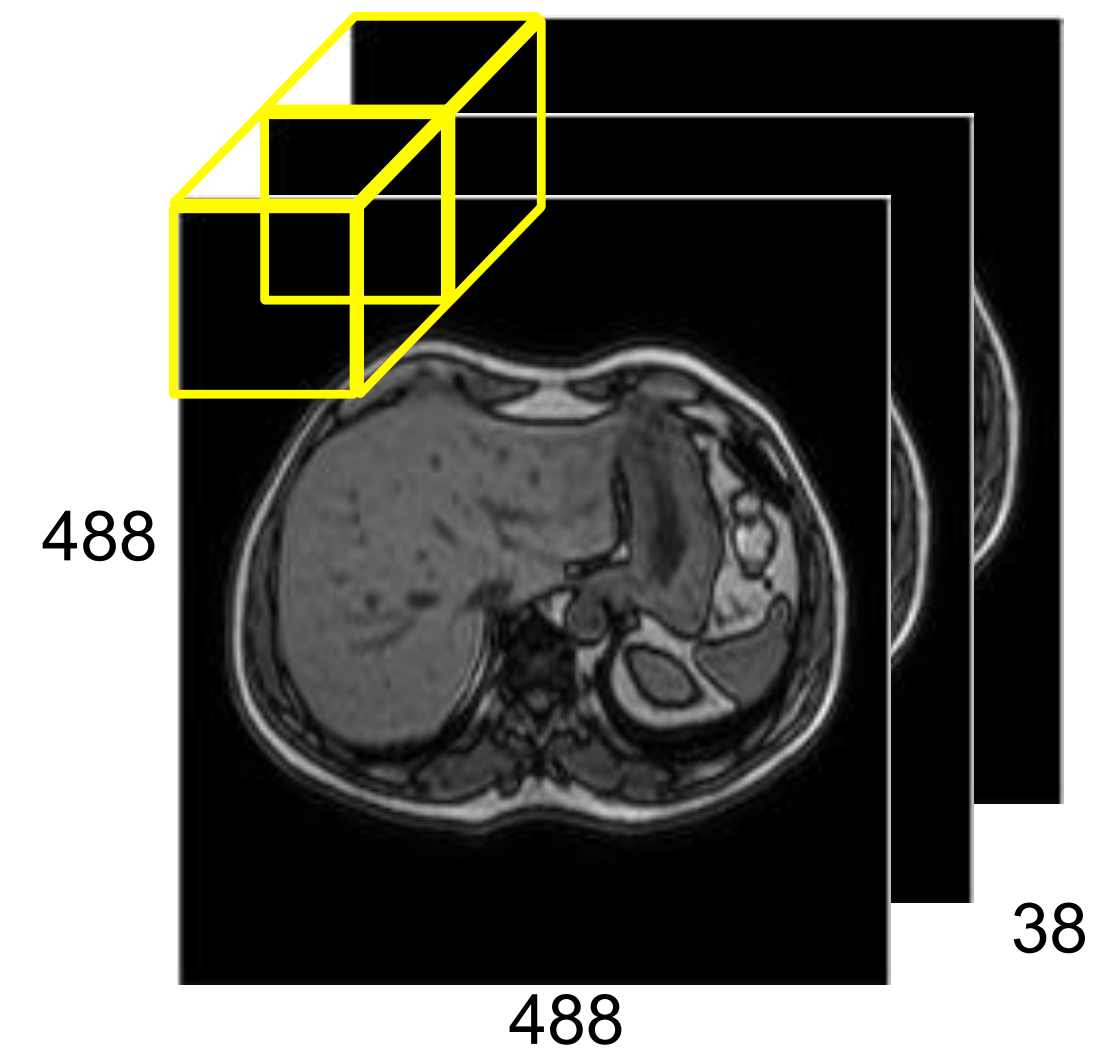
Digital/Physical
Mismatch



Resampling
Process



Digital/Physical
Match



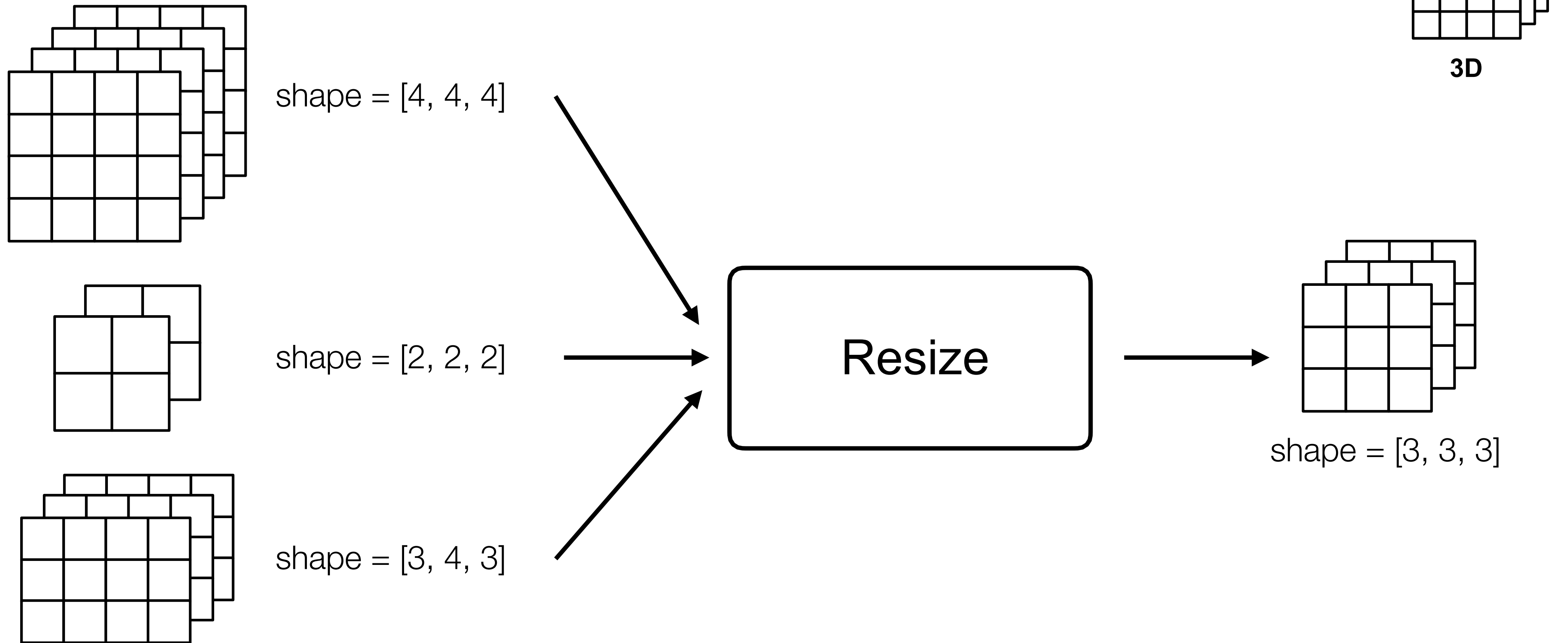
After resampling each voxel maps the information of physical space in the same way

Dataset Issues

When dealing with a dataset we can find the following issues:

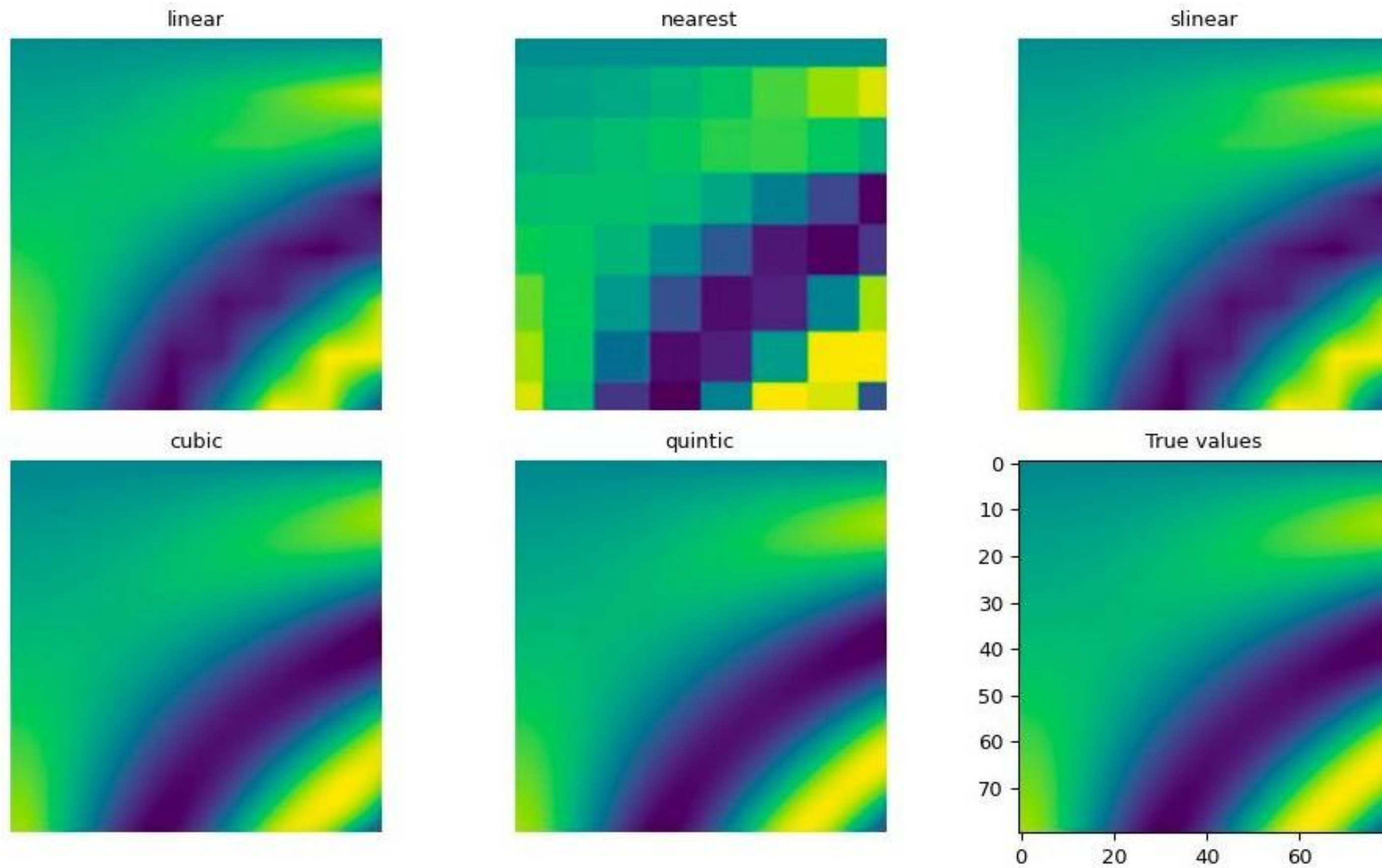
- Different pixel sizes, pixel spacing (a.k.a. slice thickness)
- Different image dimensions
- Variable contrasts
- Variable intensities
- Noise
- Artifacts

Resizing



In automatic image processing is desirable to have data with same dimensions

Interpolation



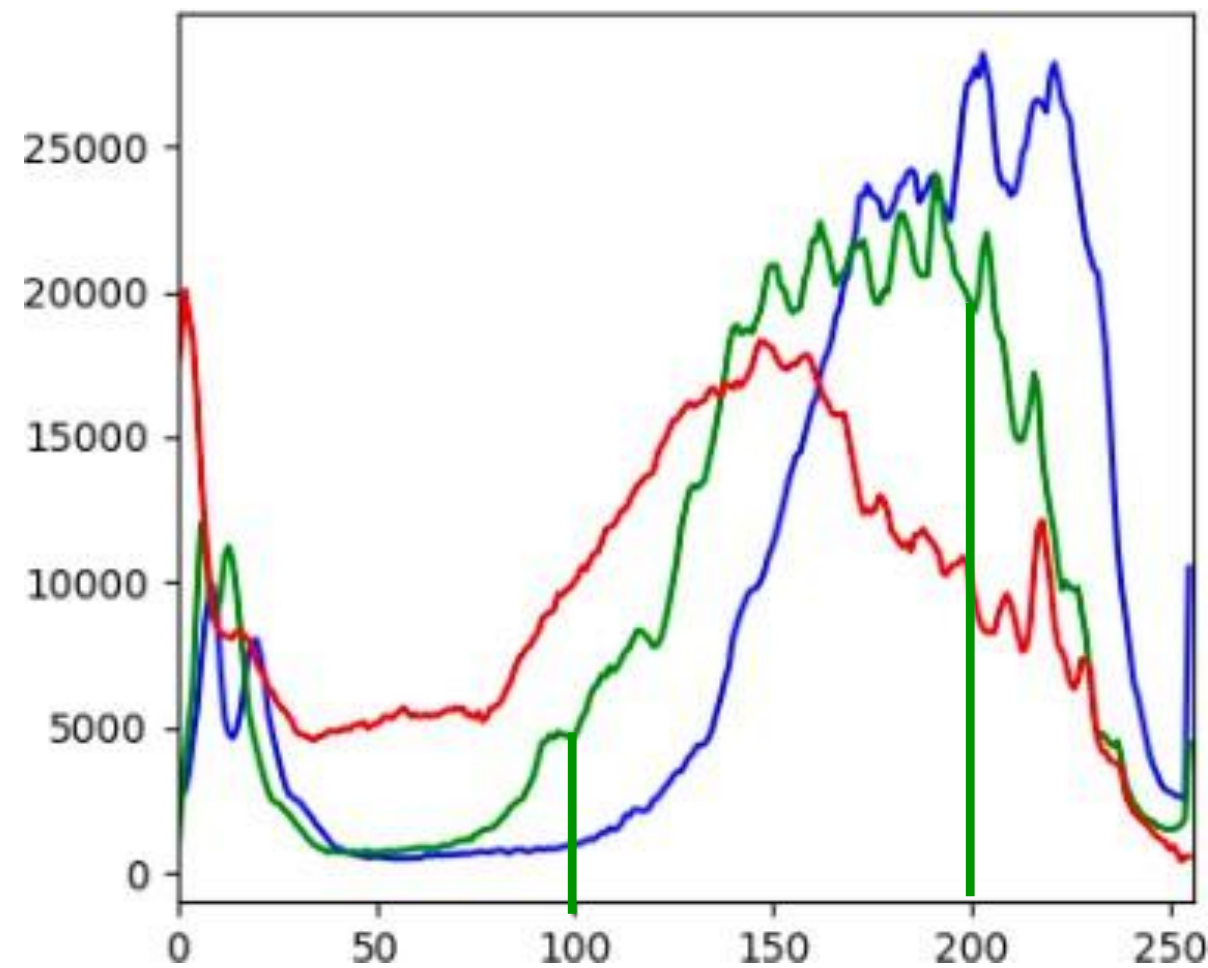
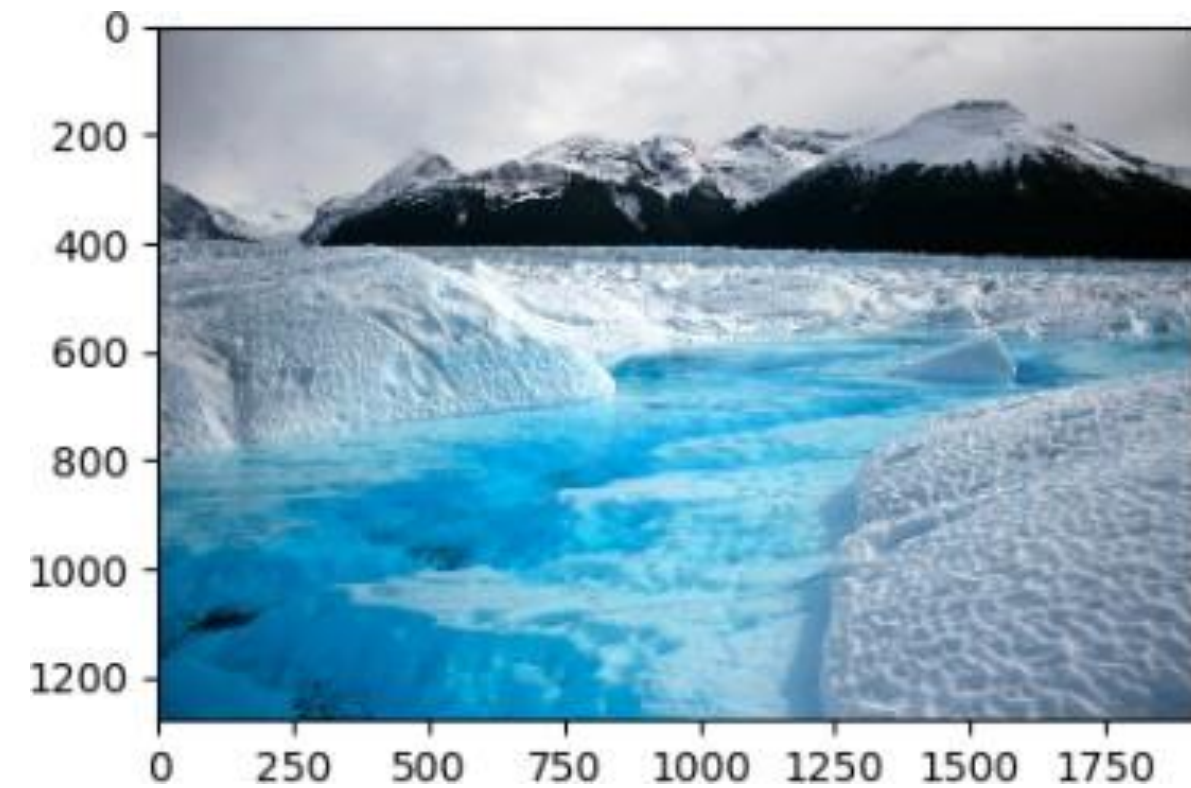
Interpolation allows correct information mapping when changing the grid dimensions

Dataset Issues

When dealing with a dataset we can find the following issues:

- Different pixel sizes, pixel spacing (a.k.a. slice thickness)
- Different image dimensions
- Variable contrasts
- Variable intensities
- Noise
- Artifacts

Histogram

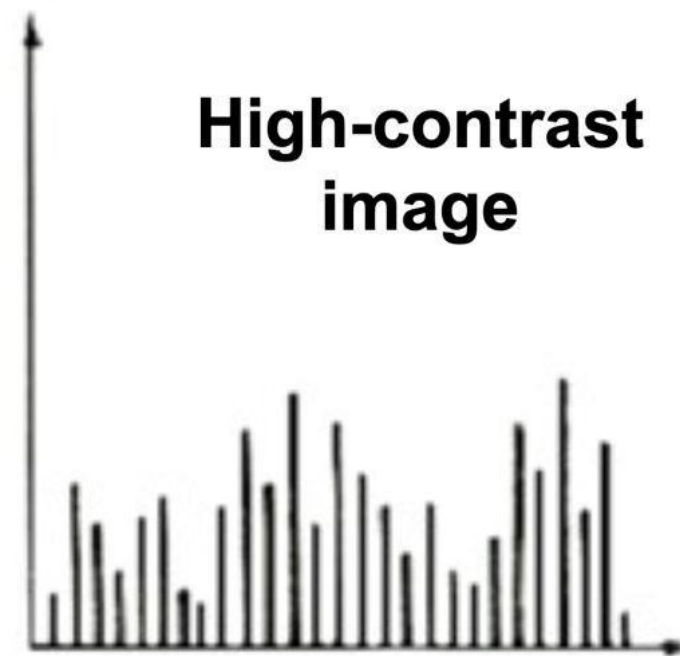
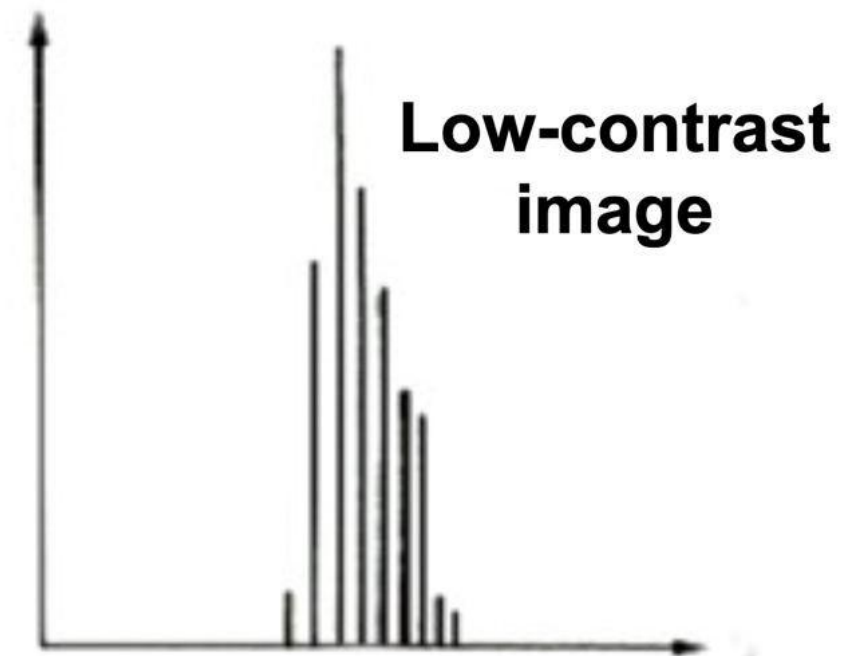
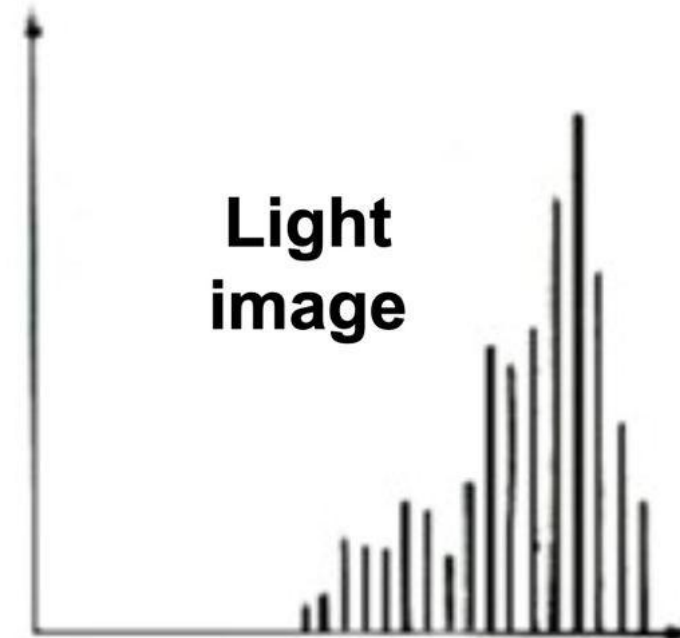
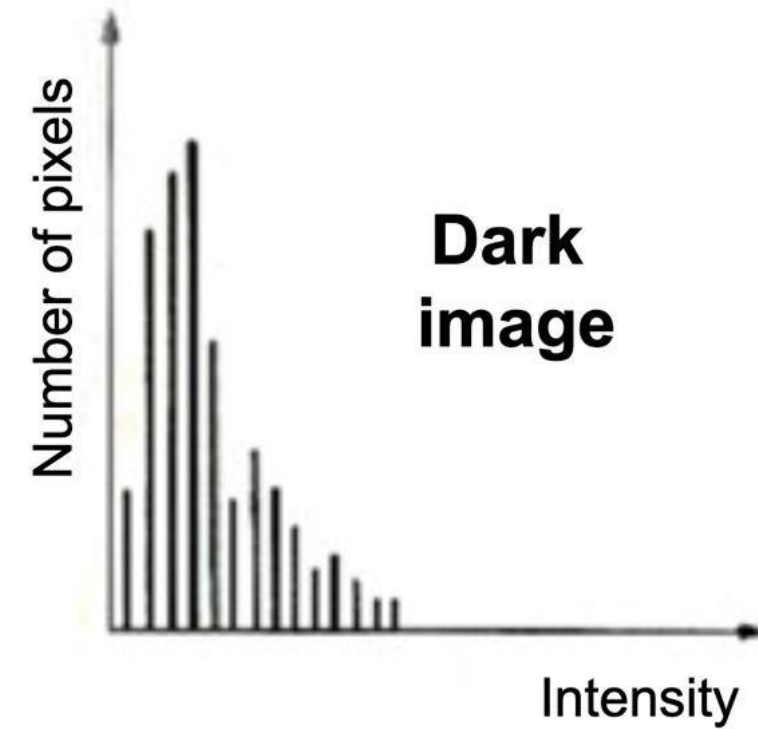
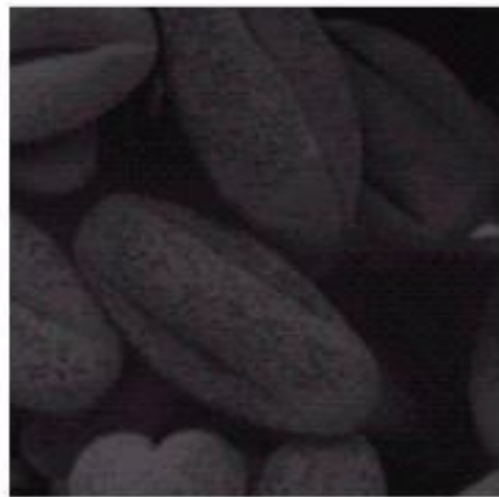


For a B-bit image f:

- Initialize 2^B bins
- Loop over all pixels x,y
- When encountering the $f(x,y) = i$ increment counter #i

A histogram represents the visual intensity distribution of the pixels across the image

Histogram

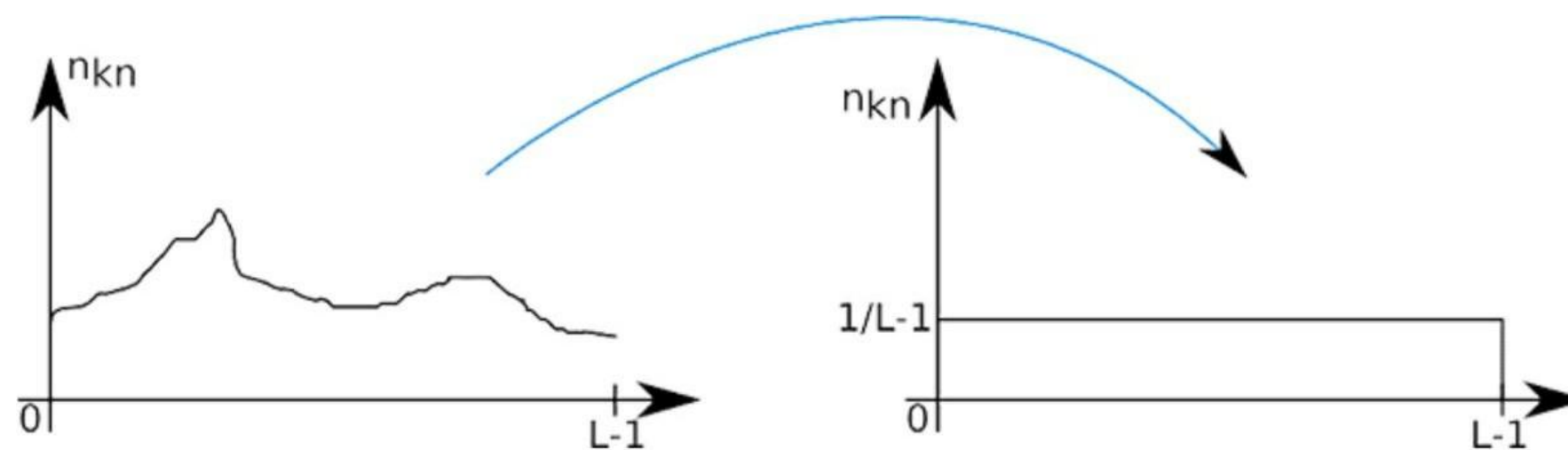


The histogram carries useful information regarding the image content

Histogram Equalization

Notice how our histogram has numerous peaks, indicating there are a good number of pixels binned to those respective buckets. With histogram equalization, our goal is to spread these pixels to buckets that don't have as many pixels binned to them.

Mathematically, what this means is that we're attempting to apply a linear trend to our cumulative distribution function (CDF)



Adaptive Histogram Equalization

When dealing with medical images it is better to employ adaptive histogram equalization strategies since we do not want to boost the noise in the input image. Input image is divided in $N \times M$ grid to then apply the equalization to each cell in the grid. Down side: it is more computationally complex



Gamma correction

When twice the number of photons hit the sensor of a digital camera, it receives twice the signal (a linear relationship). However, that's not how our human eyes work. Instead, we perceive double the amount of light as only a fraction brighter (a non-linear relationship)!

Furthermore, our eyes are also much more sensitive to changes in dark tones than brighter tones (another non-linear relationship).

In order to account for this we can apply gamma correction, a translation between the sensitivity of our eyes and sensors of a camera.

Gamma correction (a.k.a. Power Law Transform)

First, our image pixel intensities must be scaled from the range [0, 255] to [0, 1.0].

From there, we obtain our output gamma corrected image by applying the following equation:

$$O = I ^ { (1 / G) }$$

Where I is our input image and G is our gamma value. The output image O is then scaled back to the range [0, 255].

Gamma correction (a.k.a. Power Law Transform)

Gamma values < 1 will shift the image towards the darker end of the spectrum while gamma values > 1 will make the image appear lighter. A gamma value of $G=1$ will have no affect on the input image.



Gamma correction (a.k.a. Power Law Transform)

Case $G > 1$ (e.g. $G = 1.5$)

$$\text{Output} = \text{Input}^{(1/1.5)} = \text{Input}^{0.67}$$

- Dark pixel (e.g. 0.1): $\text{Output} = 0.1^{0.67} = 0.21$
- Light pixel (e.g. 0.9): $\text{Output} = 0.9^{0.67} = 0.93$

Case $G < 1$ (e.g. $G = 0.5$)

$$\text{Output} = \text{Input}^{(1/0.5)} = \text{Input}^2$$

- Dark pixel (e.g. 0.1): $\text{Output} = 0.1^2 = 0.01$
- Light pixel (e.g. 0.9): $\text{Output} = 0.9^2 = 0.81$
- Mid-value pixel (e.g. 0.5): $\text{Output} = 0.5^2 = 0.25$

Dataset Issues

When dealing with a dataset we can find the following issues:

- Different pixel sizes, pixel spacing (a.k.a. slice thickness)
- Different image dimensions
- Variable contrasts
- Variable intensities
- Noise
- Artifacts

Image Normalization: Dataset-wise Standardization

- Compute a mean volumetric image and a standard deviation volumetric image from the dataset.
- Now, normalize each volumetric image by pixel-wise subtracting the mean and dividing by the standard deviation.
- This leads to each pixel having zero mean and 1 standard deviation across the dataset.

Image Normalization: Volumetric Image-wise

Standardization

- For each volumetric image, from each pixel, subtract the mean and divide by the standard deviation computed over that volume. Do this for all volumes in the dataset independently

Normalization

- For each volumetric image, from each pixel, subtract the minimum intensity of that volume and divide by the intensity range (max - min) of that volume. Do this for all volumes in the dataset independently.
- To be robust to outlier pixels, use 1st / 2nd / 5th and 99th / 98th / 95th percentiles instead of minimum and maximum values.

Biomedical Computer Vision

March 17th, 2025

Virginia Tasso
virginia.tasso@polimi.it

